POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Rolling-Horizon $A^\star$ Planning and Nonlinear MPC for Map-Free Navigation of a Legged Robot

Project Report
062047 – Numerical Optimization for Control

Professor:
Lorenzo Mario Fagiano

Group Members:
Lorenzo Ortolani
10751686

Francesco Pedrini
10787879

Academic year:
2025-2026

Abstract: Autonomous navigation in an unknown, cluttered environment can be posed as a pair of nested optimization programs solved at every control cycle: a discrete shortest-path problem on a LiDAR-derived occupancy grid, and a continuous nonlinear program (NLP) that turns the resulting geometric reference into dynamically feasible velocity commands. Here, we analyze the map-free navigation of a legged robot, primarily a Unitree G1 humanoid, with a Unitree Go2 quadruped as the second platform. The high level plant consists in a non-linear forward kinematic dynamics, valid for each legged robot abstracted as a body-frame holonomic agent, with first-order actuator lag for modeling the low level dynamics latency, yielding a six-state, three-input discrete model; the nonlinear MPC tracker formulated in CasADi and solved by IPOPT at 8 Hz. Because that abstraction leaves the two machines with the same states and the same inputs, the stack is substantially identical on both: what changes between them is the tuning, the assumed lag and the quantitative conclusions that follow, not the formulation. The eleven cost weights of that NLP are firstly tuned by heuristics, but can be further improved by an offline calibration using a per-axis Bayesian scheme: a one-dimensional Gaussian process per weight, six closed-loop mission rollouts each, swept in order of influence with every optimum carried forward, for 67 rollouts in total; that campaign is the one part of the study it was possible to run only on the quadruped, while there was no possibility to deploy it on the humanoid for time reasons, so the reported results refer to the Go2, even if a completely analogous approach could be applied to the humanoid as well. The remaining analysis are instead mainly focused on the G1. Every design decision in the project derives from theoretical considerations and quantified outcomes: from the integration method to discretize the model to the Hessian computation method to be employed, the decision between interior-point and active-set solvers, sparse against condensed parametrisations, a smooth penalty against an exact ℓ^1 relaxation of the clearance constraint, and a terminal equilibrium constraint against none. The first-order and second-order optimality conditions are verified on recorded missions rather than assumed. Regarding the Bayesian optimization campaign, every claim in the performance analysis is produced by driving the deployed CasADi/IPOPT tracker in closed loop inside an offline harness, over different synthetic worlds and many parametric studies.

Key-words: Nonlinear Model Predictive Control, Interior-Point Optimization, Rolling-Horizon Planning, $A^\star$ Search, Map-Free Legged Robot Navigation, Bayesian Optimization

Contents

Acronyms

AD	algorithmic differentiation
AMO	Adaptive Motion Optimization
DARE	discrete algebraic Riccati equation
EKF	extended Kalman filter
FHOCP	finite-horizon optimal control problem
GP	Gaussian process
IK	inverse kinematics
IMU	inertial measurement unit
KKT	Karush–Kuhn–Tucker
LICQ	linear independence constraint qualification
LiDAR	light detection and ranging
LTI	linear time-invariant
MPC	model predictive control
NLP	nonlinear program
PD	proportional–derivative
PPO	proximal policy optimization
QP	quadratic program
RL	reinforcement learning
ROS	Robot Operating System
SLAM	simultaneous localization and mapping
SQP	sequential quadratic programming

Notation

All physical quantities carry SI units. Vectors are columns; $\|\cdot\|$ without subscript is the Euclidean norm. Subscript k indexes a node inside the prediction horizon, subscript j an obstacle. The symbol $\lambda_{\min}(\cdot)$ denotes the smallest eigenvalue of its argument, and is not related to the multipliers λ.

State, input and model

x	state	$\tau_{v_x}, \tau_{v_y}, \tau_\omega$	actuator time constants [s]
u	commanded body twist	$\nu_{v_x}, \nu_{v_y}, \nu_\omega$	discrete lag coefficients
p_x, p_y, ψ	world-frame pose [m], [rad]	Δt	sampling time [s]
v_x, v_y, ω	body twist [m/s], [rad/s]	N	prediction horizon [steps]
t	continuous time [s]	N_c	control horizon [steps]

Environment and reference

$\mathcal{P}_k$	LiDAR point cloud at cycle k	δ	occupancy grid resolution [m]
$P(c)$	occupancy of cell c	σ	Gaussian inflation width [m]
x_g	goal position [m]	P_{thr}	occupancy hard-obstacle threshold
v_{ref}	cruise speed [m/s]	π_k	$A^\star$ path at cycle k

Optimization program

$z = (X, U)$	decision variables	λ, μ	KKT multipliers (eq., ineq.)
J	objective	μ_b	interior-point barrier parameter
Q, R, R_{jerk}	tracking, effort, rate weights	ρ_T	terminal weight multiplier
d_{safe}	imposed clearance [m]	s_{jk}	constraint slack [m]
$\gamma(k)$	robust back-off [m]	θ	cost-weight vector
ε_d	distance regulariser	ε_M	machine precision
$W_{\text{obs}}, \alpha_{\text{obs}}, r_{\text{obs}}$	obstacle penalty height, steepness, radius	ρ_s	slack penalty weight

Locomotion layer

$q, \dot{q}$	joint positions, velocities	β	gait duty factor
$\mathcal{T}$	joint torques	ζ_i	stride phase of leg i
K_P, K_D	joint PD gains	$T_{\text{str}}, T_{\text{st}}, T_{\text{sw}}$	stride, stance, swing durations [s]
p_i^{f}	foot position of leg i [m]	L_i	step length of leg i [m]

1. Introduction

A mobile robot that must reach a goal in an environment it has never seen has no global map to plan on: at every instant it knows only what its sensors reported in the last few metres. The classical answer is to decouple the problem into a geometric planner and a low-level tracker, but if the two layers ignore each other the planner produces paths the actuators cannot execute, and the tracker produces commands the geometry does not allow. Model Predictive Control (MPC) resolves the conflict by construction: it optimizes a finite sequence of future inputs subject to the actual dynamics and the actual constraints, and re-optimizes at every cycle [11]. The price is computational for the motion planner point of view: a nonlinear program must be solved to sufficient accuracy inside the sampling period, every period, on the robot. However, this rolling horizon path planning, allows the robot to avoid simultaneously localize and map the enviroment. The presence of only a localization module throguh an EKF and the lack of complete SLAM, enables less computationally expensive navigation stack, which is an important contribution of this report.
The goal is to analyze legged robotics navigation stack, such as a Unitree Go2 dog robot or a Unitree G1 humanoid robot. The stack is deployed and tested entirely on the G1 robot and previously developed on a Unitree Go2 quadruped, whose locomotion layer, the CHAMP gait controller [5] on the simulated robot, the vendor's sport-mode controller on the physical one, exposes a body-frame velocity interface; the same code has since been ported to a Unitree G1 humanoid walking under a reinforcement-learning locomotion policy [7], which is what makes the "robot-agnostic" claim testable, due to the simple high level holonomic kinematic model derived from both systems. Section 2.3 describes those two locomotion layers and the abstraction that makes them interchangeable. Two programs are solved at every planning cycle and are the object of the analysis:

- a *discrete* shortest-path program, an $A^\star$ search over a Gaussian occupancy grid rebuilt from the current LiDAR scan, whose solution is a geometric reference and whose rolling-horizon target is the intersection of the ray to the global goal with the grid boundary;
- a *continuous* nonlinear program, a six-state, three-input MPC with actuator-lag dynamics, a smooth obstacle barrier and box-constrained inputs, formulated once symbolically in CasADi [2] and solved by the interior-point method IPOPT [15] at 8 Hz.

On top of these two runtime programs sits a third, solved once offline: the *black-box* program of choosing the eleven cost weights of the MPC. Weight tuning is the practical bottleneck of any deployed MPC, since depending on the enviroment, MPC performance can vary significantly. The problem is that the closed-loop performance of the MPC is a gray-box function of unknown parameters, namely the weights for each cost term. The automatic weights tuning problem is implemented with Gaussian-process surrogates [10, 14] for finding the optimal weight vector $\hat{\theta}$ of the MPC cost function.

2. Problem analysis

2.1. System overview

The stack is a chain of ROS 2 nodes that closes the loop between a 3-D LiDAR and the body-frame velocity interface of the locomotion controller:

1. an **odometry bridge** that converts the extended-Kalman-filter (EKF) pose estimate into a single pose topic and differentiates it, with an exponential filter, into a body-frame velocity estimate (the robot does not publish a usable body-frame twist);
2. a **mapping module** that rebuilds a local Gaussian occupancy grid from the newest scan at 1 Hz and maintains a sparse world-frame cell map of confirmed obstacle cells, on which the memory-augmented target rule of Section 5.6.2 plans; its purpose is precisely to repair the failure mode analysed in Section 5.6.1;
3. the $A^\star$ **local planner**, which solves the discrete program of Section 3.1 on that grid;
4. the **nonlinear MPC tracker**, which solves the NLP of Section 3.3 at 8 Hz and publishes a setpoint;

Two design decisions shape everything that follows. First, the locomotion layer is treated as an ideal velocity-tracking device with a time constant: we never model legs, contacts or gait, and the robot is abstracted as a planar holonomic agent, Sections 2.2 and 2.3 describe what that layer actually does on each of the two platforms, and why the same abstraction survives the difference between them. Second, the map is *local and volatile*. The occupancy grid covers a fixed 10×10 m window centred on the robot and is rebuilt from scratch at every planning cycle, so the planning problem is genuinely map-free, and the geometric reference the MPC tracks is only valid for one grid width ahead. Section 5.6.1 measures what this choice costs, and Section 5.6.2 what it takes to repair it.

2.2. The controlled platforms

The same stack drives two robots whose mechanics could hardly be more different: a Unitree Go2 quadruped (Fig. 1(a)) and a Unitree G1 humanoid (Fig. 1(b)). The Go2 has twelve actuated joints, three per leg; the G1 has twenty-nine, of which the locomotion layer of Section 2.3.2 drives fifteen, the twelve leg joints and the three waist joints, while the arms are held at a fixed posture. Neither joint set is ever addressed by the high level trajectory tracking MPC controller. Both platforms are commanded through the same channel, a body-frame twist $u = (v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, \omega^{\mathrm{cmd}})$, and both carry a 3-D LiDAR whose returns are the only exteroceptive input of the stack.

(a) Unitree Go2 quadruped

photograph of the G1 humanoid
(add `Images/G1.jpeg`)

(b) Unitree G1 humanoid

Figure 1: The two platforms on which the same navigation stack is deployed. (a) The Go2 quadruped, with its LiDAR mounted on the head. (b) The G1 humanoid, to which the stack was ported without changing the optimization program: only the constants of the actuator-lag model (12) and the command mapping of Section 2.3.2 differ.

Table 1: The two platforms as the controller sees them. The rows above the first rule are properties of the machine, those between the rules are properties of the locomotion layer that receives the MPC command, and those below are the ones that enter the optimization program: the prediction model (12), the admissible input set and the horizon. The three time constants are the same assumed values on both machines (Section 2.5), never identified on either, which is the concrete form the "robot-agnostic" claim takes.

	Unitree Go2	Unitree G1
Morphology	quadruped	humanoid
Actuated joints	12 (3 per leg)	29 (15 driven by the gait policy)
Exteroception	Unitree L1 LiDAR	Livox MID-360 LiDAR
Locomotion layer	CHAMP gait controller (vendor sport mode on hardware)	AMO reinforcement-learning policy
Layer update rate	200 Hz	50 Hz
Command interface	body-frame twist u	body-frame twist u, remapped (Section 2.3.2)
Abstraction used by the MPC	first-order velocity lag, $\tau_{v_x} = 0.12$ s, $\tau_{v_y} = \tau_\omega = 0.10$ s	
Admissible input set	$v_x \in [-0.20, 0.40]$ m/s, $\lvert v_y \rvert \le 0.20$ m/s, $\lvert \omega \rvert \le 0.40$ rad/s	
Horizon	$N = 15$, $\Delta t = 0.35$ s (5.25 s of preview), re-solved at 8 Hz	

2.3. Low-level locomotion layers

The controller analysed in this report never commands a joint. Its decision variable is a body-frame twist, and between that twist and the ground sits a platform-specific layer that turns it into joint commands at 200 Hz on the Go2 and 50 Hz on the G1. Only two properties of that layer reach the optimization problem: the twist is tracked with a first-order lag rather than instantaneously, and the tracking is fast enough, relative to the 8 Hz MPC cycle, for the two layers to be designed separately. This section describes the two locomotion controllers to the depth needed to justify those two properties and no further. Throughout, $q \in \mathbb{R}^{n_j}$ denotes the vector of joint positions ($n_j = 12$ on the Go2, $n_j = 29$ on the G1), $\mathcal{T} \in \mathbb{R}^{n_j}$ the vector of joint torques, and $K_P, K_D \succ 0$ the diagonal proportional and derivative gain matrices of the joint-level loop; the symbol τ is reserved for the actuator time constants of Section 2.5.

2.3.1 Model-based gait generation: CHAMP on the Go2

CHAMP [5] is an open-source implementation of the hierarchical trot controller developed for the MIT Cheetah [4, 6]. It is a purely feedforward pipeline: a pattern modulator imposes the footfall sequence, a trajectory planner turns each leg's phase into a foot position, inverse kinematics (IK) turns that position into joint angles, and a joint-level impedance loop tracks them. CHAMP is the locomotion layer of the simulated robot.

Pattern modulation. Let t denote time. Each leg $i \in \{\mathrm{LF}, \mathrm{RF}, \mathrm{LH}, \mathrm{RH}\}$ carries a normalised stride clock

$$\zeta_i(t) = \left(\frac{t}{T_{\mathrm{str}}} - \zeta_i^0 \right) \bmod 1 \in [0, 1), \qquad T_{\mathrm{str}} = T_{\mathrm{st}} + T_{\mathrm{sw}}, \tag{1}$$

where T_{st} and T_{sw} are the stance and swing durations and ζ_i^0 is the phase offset of leg i. Leg i is in stance while $\zeta_i < \beta$ and in swing otherwise, with duty factor $\beta = T_{\mathrm{st}}/T_{\mathrm{str}}$. The deployed configuration uses $T_{\mathrm{st}} = T_{\mathrm{sw}} = 0.25$ s, hence $T_{\mathrm{str}} = 0.5$ s and $\beta = 0.5$, and $\zeta^0 = (0, \frac{1}{2}, \frac{1}{2}, 0)$: the diagonal pairs move together, which is the trot.

Where the command enters. The commanded twist does not act on the legs directly; it acts on the *step geometry*, through the Raibert foot-placement heuristic [9]: a foot is placed half a step ahead of its nominal position, so that sweeping backwards during the stance interval T_{st} displaces the trunk by exactly the commanded amount. Let $\bar{p^{\mathrm{f}}}_i \in \mathbb{R}^3$ be the nominal (zero-command) position of foot i in the body frame. Translating the nominal stance by the half-step $\frac{1}{2}T_{\mathrm{st}}\,(v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, 0)^\top$ and rotating it about the vertical axis by the half-stance yaw χ_g gives the foot displacement

$$\Delta p_i^{\mathrm{f}} \;=\; R_z(\chi_g)\Big(\bar{p^{\mathrm{f}}}_i + \tfrac{1}{2}T_{\mathrm{st}}\,(v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, 0)^\top\Big) - \bar{p^{\mathrm{f}}}_i, \qquad \chi_g = 2\sin\Big(\tfrac{1}{4}T_{\mathrm{st}}\,\omega^{\mathrm{cmd}}\Big) \approx \tfrac{1}{2}T_{\mathrm{st}}\,\omega^{\mathrm{cmd}}, \tag{2}$$

with $R_z(\cdot)$ the elementary rotation about the vertical axis.[1] The step length and the heading of the foot trajectory of leg i follow as

$$L_i = 2\big\|\Delta p_i^{\mathrm{f}}\big\|_2, \qquad \chi_i = \operatorname{atan2}\big(\Delta p_{i,y}^{\mathrm{f}},\, \Delta p_{i,x}^{\mathrm{f}}\big), \tag{3}$$

the factor 2 turning the half-step into the full stride of the foot. Equations (2) and (3) are the whole of the command path: the three numbers the MPC publishes are compressed into one length and one angle per leg, and nothing else of the command reaches the robot.

Foot trajectories and impedance. With phase and step length fixed, each foot follows a planar curve in its own trajectory frame, rotated by χ_i and added to the nominal stance: $p_i^{\mathrm{f}} = \bar{p^{\mathrm{f}}}_i + (\,\xi\cos\chi_i,\ \xi\sin\chi_i,\ \eta\,)^\top$, where ξ and η are the longitudinal and vertical coordinates of the curve. In stance, parametrised by the normalised stance phase $\zeta_i^{\mathrm{st}} = \zeta_i/\beta \in [0, 1)$, the foot sweeps backwards along a shallow arc that presses into the ground by h_{st},

$$\xi = \tfrac{L_i}{2}\left(1 - 2\zeta_i^{\mathrm{st}}\right), \qquad \eta = -h_{\mathrm{st}}\cos\left(\frac{\pi\,\xi}{L_i}\right); \tag{4}$$

in swing, parametrised by $\zeta_i^{\mathrm{sw}} = (\zeta_i - \beta)/(1 - \beta) \in [0, 1)$, it follows a Bézier curve of degree 11 through twelve control points $b_j \in \mathbb{R}^2$,

$$(\xi, \eta) = \sum_{j=0}^{11} \binom{11}{j} \left(\zeta_i^{\mathrm{sw}}\right)^j \left(1 - \zeta_i^{\mathrm{sw}}\right)^{11-j} b_j, \tag{5}$$

[1] The implementation obtains χ_g from the tangential half-step $\frac{1}{2}T_{\mathrm{st}}\,\omega^{\mathrm{cmd}} d_c$, with d_c the horizontal distance from the trunk centre to the nominal foot; d_c cancels, leaving (2). The small-angle form on the right is the yaw swept in half a stance interval, as expected.

whose horizontal spread is scaled by L_i and whose vertical spread is scaled by the swing height h_{sw}. Figure 2 collects these quantities on a single leg. The deployed gait uses $h_{\mathrm{sw}} = 0.04$ m, $h_{\mathrm{st}} = 0.01$ m and a nominal trunk height $h_0 = 0.225$ m. A closed-form IK then maps each foot position to the three joint angles of its leg, $q_i^{\mathrm{ref}} = \mathrm{IK}_i(p_i^{\mathrm{f}})$, and a proportional–derivative (PD) law closes the loop at the joints,

$$\mathcal{T}_i = K_P\big(q_i^{\mathrm{ref}} - q_i\big) + K_D\big(\dot{q}_i^{\mathrm{ref}} - \dot{q}_i\big) \;=\; \boldsymbol{J}_i(q_i)^{\top} F_i, \tag{6}$$

where $\boldsymbol{J}_i$ is the foot Jacobian of leg i, set in bold to distinguish it from the MPC cost J and from the tuning objective $\mathcal{J}$ and F_i the equivalent force at the foot. The PD loop is in joint space and realises a virtual spring–damper at the foot, of Cartesian stiffness $\boldsymbol{J}_i^{-\top} K_P \boldsymbol{J}_i^{-1}$, so the leg absorbs terrain irregularity and impact by programmable compliance rather than by contact sensing or force control. This is what makes an open-loop gait generator survive on a real robot.

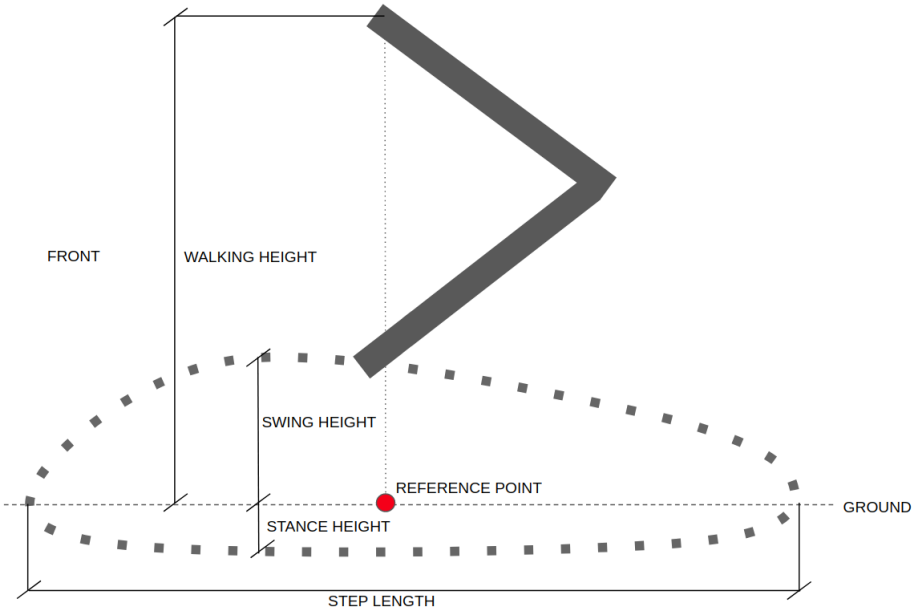

Figure 2: Geometry of the CHAMP gait on a single leg, seen from the side. The dotted closed curve is the cycle described by the foot in its trajectory frame: the upper branch is the swing Bézier (5), which clears the ground by the swing height h_{sw}, and the lower branch is the stance arc (4), which presses into the ground by the stance height h_{st} and sweeps backwards to carry the trunk forward. The horizontal extent of the cycle is the step length L_i of (3), the only channel through which the commanded twist reaches the leg; the reference point on the ground line is the nominal stance position $\bar{p^{\mathrm{f}}}_i$ about which the Raibert half-step (2) displaces the cycle, and the walking height is the nominal trunk height h_0. Adapted from the CHAMP documentation [5].

Low level joint controllers modelled as a lag. Three consequences propagate upstream, and they are the reason the model of Section 2.5 has the form it has. First, the twist enters only through the step geometry (3): no measured trunk velocity is fed back, so the layer is a feedforward map and the trunk velocity approaches the commanded one over roughly one stride rather than immediately. Second, that step geometry is re-evaluated only once per stride, so a command that changes faster than $1/T_{\mathrm{str}} = 2$ Hz is averaged, not tracked; a first-order lag with $\tau_{v_x} = 0.12$ s reaches 98% of its final value in $4\tau_{v_x} \approx T_{\mathrm{str}}$, which is why the one-parameter fit of Section 2.5 describes the layer as well as it does. Third, the gait applies its own saturation, $|v_x^{\mathrm{cmd}}| \le 0.3$ m/s, $|v_y^{\mathrm{cmd}}| \le 0.25$ m/s and $|\omega^{\mathrm{cmd}}| \le 0.5$ rad/s in the deployed configuration.

2.3.2 Learned whole-body control: the AMO policy on the G1

On the humanoid the same interface is produced by a neural network instead of a gait generator. The deployed controller is AMO (Adaptive Motion Optimization) [7], a reinforcement-learning (RL) policy trained in simulation for the G1 and its twenty-nine degrees of freedom (DoF). AMO is deliberately *not* a walking controller: it is a loco-manipulation policy, trained to track torso height, torso orientation and upper-body targets while walking, on a hybrid dataset in which an offline trajectory-optimization module supplies whole-body references that are kinematically consistent with the commanded end-effector motion. The navigation stack exercises only

its locomotion subset, the arms are held at the default posture and only the base command is driven, but the policy is the one trained for the larger task, which is what makes the same platform usable for manipulation later without changing the layer the MPC talks to.

Structure. At each tick k of the 50 Hz policy loop, the network maps an observation to an action,

$$a_k = \pi(o_k) \in \mathbb{R}^{15}, \qquad o_k = \big(\, \omega_k^{\mathrm{b}},\ \hat{g}_k,\ \tilde{u}_k,\ q_k - q^{\mathrm{def}},\ \dot{q}_k,\ a_{k-1} \,\big), \tag{7}$$

where $\omega_k^{\mathrm{b}} \in \mathbb{R}^3$ is the body angular velocity, $\hat{g}_k \in \mathbb{R}^3$ the gravity direction expressed in the body frame (the tilt an inertial measurement unit (IMU) provides without a global heading), $q_k, \dot{q}_k$ the measured joint positions and velocities of the 23 observed joints, q^{def} the default posture, a_{k-1} the previous action, and $\tilde{u}_k$ the command. The action is not a torque: it is an offset on the default posture, which the same joint-level PD law as (6) converts into torque,

$$q_k^{\mathrm{ref}} = q^{\mathrm{def}} + s_a\, a_k, \qquad \mathcal{T}_k = K_P\big(q_k^{\mathrm{ref}} - q_k\big) + K_D\big(0 - \dot{q}_k\big), \tag{8}$$

with s_a a fixed action scale and a zero reference joint velocity. The learned part of the controller is therefore exactly the map from observation to PD setpoint; the stabilising loop underneath it is the same fixed impedance as on the quadruped. The policy π is obtained by proximal policy optimization (PPO) [13], maximising the expected discounted return $\mathbb{E}\big[\sum_k \gamma^k \mathcal{R}_k\big]$ with $\gamma \in (0,1)$ and a stage reward $\mathcal{R}_k$ that pays for tracking the commanded twist and penalises posture deviation, joint effort and action rate; masses, friction, gains and latencies are randomised during training so that the fixed policy transfers to hardware [12].

2.3.3 The cascade and its abstraction seam

Fig. 3 places the two locomotion layers in the control structure they belong to. The stack is a cascade of four loops whose rates are separated by roughly a decade each: the $A^\star$ reference is recomputed at 1 Hz, the MPC at 8 Hz, the locomotion layer at 50–200 Hz, and the joint PD loop at the motor-driver rate. That separation is the standard justification for designing each layer against a static abstraction of the one below it, and the abstraction used here is deliberately minimal: three decoupled first-order lags, Eq. (12), whose time constants are chosen in Section 2.5. Both locomotion layers, the hand-designed one and the learned one, are approximated by the same small set of numbers.

2.4. Environment representation

Let $\mathcal{P}_k = \{p_i \in \mathbb{R}^3\}_{i=1}^{N_k}$ be the LiDAR point cloud available at cycle k, projected on the plane. The grid is a square of side $2h_w$ ($h_w = 5$ m) centred on the robot position, discretised with resolution $\Delta = 0.25$ m into $n_c = 2h_w/\Delta = 40$ cells per axis. The occupancy probability of cell c is a Gaussian tail of the distance to the nearest return,

$$P(c) \;=\; 1 - \Phi\!\left(\frac{d_{\min}(c, \mathcal{P}_k)}{\sigma}\right), \qquad d_{\min}(c, \mathcal{P}_k) = \min_{p \in \mathcal{P}_k} \|c - p\|_2, \tag{9}$$

with Φ the standard normal cumulative distribution function,

$$\Phi(x) \;=\; \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-t^2/2}\, \mathrm{d}t \;=\; \frac{1}{2}\left(1 + \operatorname{erf}\!\left(\frac{x}{\sqrt{2}}\right)\right) \in (0,1), \tag{10}$$

and $\sigma = 0.15$ m. Equation (9) is an *inflation* model: it is a smooth, monotone map from clearance to cost, whose one parameter σ sets the width of the obstacle halo. Cells with $P(c) \geq P_{\mathrm{thr}} = 0.4$ are hard obstacles. The left panel of Fig. 4 shows the trade-off governed by σ: a small σ leaves doorways open but hugs walls, a large one closes gaps the robot could physically cross.

Because the grid is rebuilt from scratch, its construction cost is the dominant cost of the planning layer, $O(n_c^2\,|\mathcal{P}_k|)$; vectorised over 40×40 cells and $\sim$180 returns it measures 4–5 ms on average together with the $A^\star$ search (Section 5.1), which at the 1 Hz replanning rate is negligible against the 8 Hz MPC.

2.5. Plant model and state-space formulation

The MPC model must predict where the robot will be. Commanding a legged platform a body-frame velocity does not produce that velocity instantaneously: the locomotion controller responds with a measured time constant of $\tau \approx 0.10$–0.15 s, which over a 5 s horizon accumulates into a prediction error of several tens of

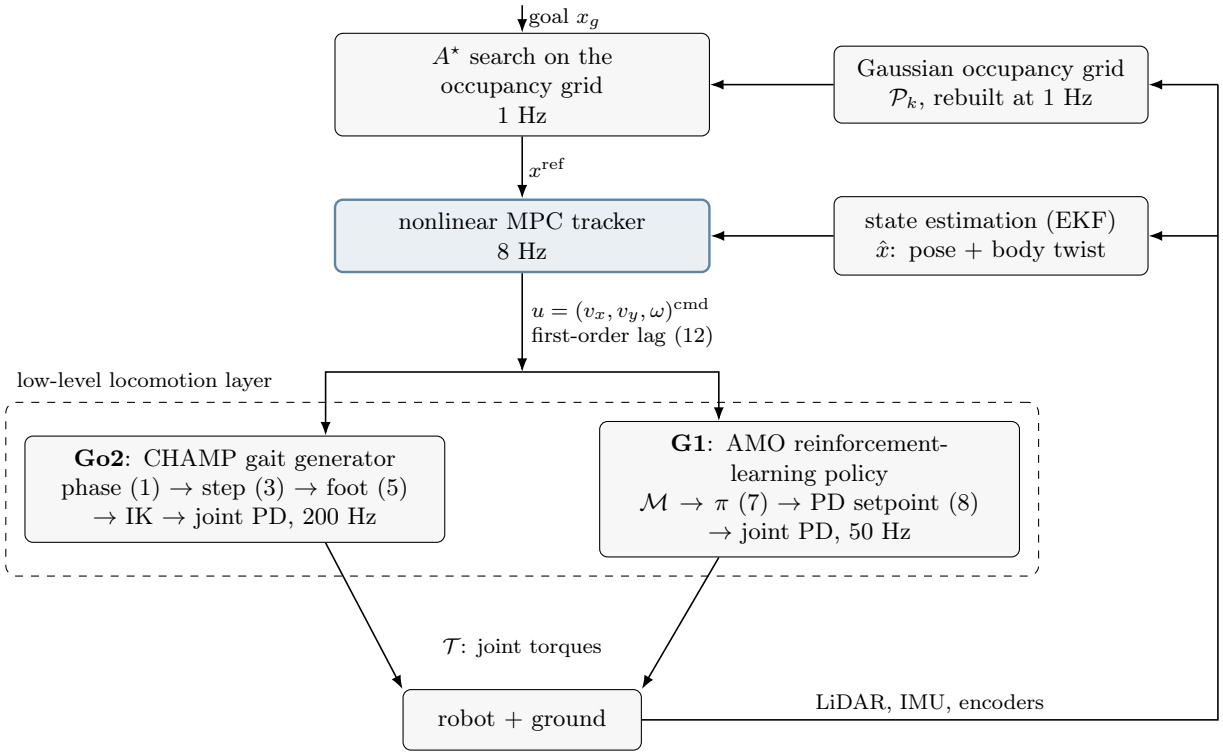

Figure 3: Cascaded structure of the controller, from the goal down to joint torques. The rates fall by roughly a decade at every seam (1 Hz, 8 Hz, 50–200 Hz, motor-driver rate), which is what allows the layers to be designed separately, and the two branches inside the dashed box, a hand-designed gait generator and a learned policy, are interchangeable precisely because they present the same twist interface u.

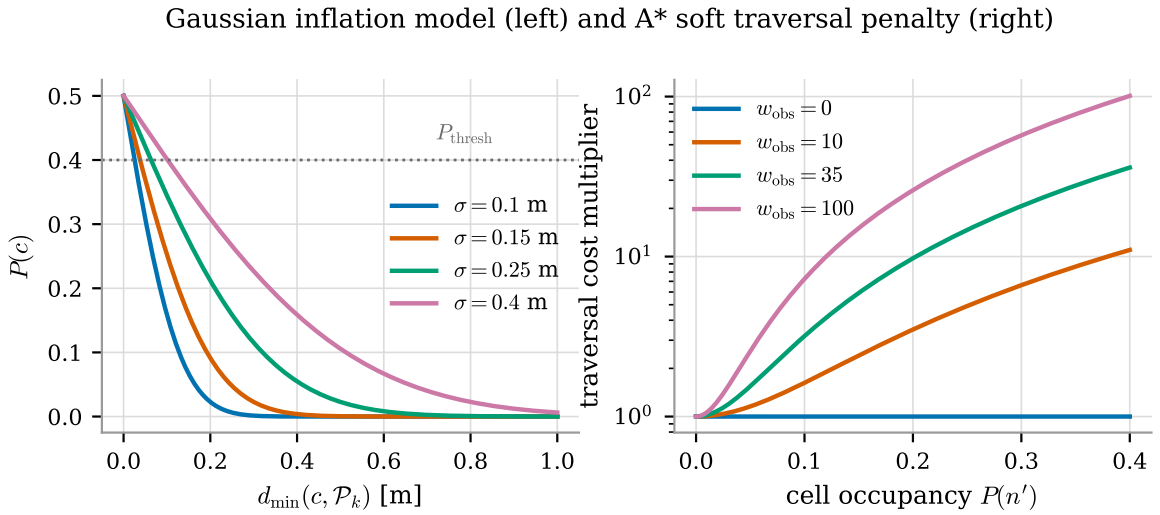

Figure 4: Left: the Gaussian inflation model (9) for four values of σ, with the hard threshold $P_{\text{thr}} = 0.4$. Right: the soft traversal-cost multiplier $1 + w_{\text{obs}}(P/P_{\text{thr}})^2$ of the $A^\star$ edge cost (15), which is what actually keeps the path away from walls; the deployed value is $w_{\text{obs}} = 35$.

centimetres if neglected. The model therefore augments the usual planar kinematic state with the three velocity states, giving

$$x = \begin{bmatrix} p_x & p_y & \psi & v_x & v_y & \omega \end{bmatrix}^\top \in \mathbb{R}^6, \qquad u = \begin{bmatrix} v_x^{\mathrm{cmd}} & v_y^{\mathrm{cmd}} & \omega^{\mathrm{cmd}} \end{bmatrix}^\top \in \mathbb{R}^3, \tag{11}$$

where (p_x, p_y) and ψ are the world-frame pose and (v_x, v_y, ω) the body-frame twist actually realised by the platform. In continuous time the actuator behaves as three decoupled first-order lags driven by the commands,

$$\dot{v}_x = \frac{u_1 - v_x}{\tau_{v_x}}, \qquad \dot{v}_y = \frac{u_2 - v_y}{\tau_{v_y}}, \qquad \dot{\omega} = \frac{u_3 - \omega}{\tau_\omega}, \tag{12}$$

while the pose obeys the holonomic body-to-world rotation

$$\dot{p}_x = v_x \cos\psi - v_y \sin\psi, \qquad \dot{p}_y = v_x \sin\psi + v_y \cos\psi, \qquad \dot{\psi} = \omega. \tag{13}$$

The three constants are design choices, not identified quantities. No step-response experiment was run on either platform, and the simulation-error identification that would produce them is a different class of problem (optimal system identification) that is deliberately left outside the scope of this report. What was done instead is to pick values of the right order of magnitude, namely the settling time of the low-level locomotion loop, which runs one decade faster than the $\Delta t = 0.35$ s of the tracker, and to check that the closed loop is insensitive to the choice: $\tau_{v_x} = 0.12$ s, $\tau_{v_y} = \tau_\omega = 0.10$ s.

The lateral and the yaw constant are held equal because nothing in the available data distinguishes them. Both channels are produced by the same low-level layer from the same twist command (CHAMP's body-velocity controller on the Go2, the AMO policy's tracking head on the G1), whereas forward travel is the axis the gait is designed around and is given the slightly slower response. Setting $\tau_{v_y} = \tau_\omega$ is therefore an admission that only two responses are being modelled, not three; the third symbol is kept in the notation because the axes are conceptually distinct and because an identification under physics would very likely separate them. At the horizon used here the choice is in any case immaterial: at $\Delta t = 0.35$ s the two discrete lag coefficients differ by 0.9459 against 0.9698, a gap far below the prediction error of Section 3.2.

Model (12)–(13) is holonomic: unlike a differential-drive vehicle, the platform can command a lateral velocity, so no nonholonomic constraint appears and the input set is a box. This is what makes the resulting NLP comparatively benign, the nonlinearity is confined to the rotation $R(\psi)$ and to the obstacle distances.

2.6. Navigation problem statement

Let $x_g \in \mathbb{R}^2$ be the goal. The navigation task is the receding-horizon optimal control problem

$$\begin{aligned}
\min_{u_0,\dots,u_{N-1}} \quad & J(x_{0:N}, u_{0:N-1}, \mathcal{P}_k, \theta) && && \text{(14a)}\\
\text{s.t.} \quad & x_{t+1} = f(x_t, u_t), && t = 0, \dots, N-1, && \text{(14b)}\\
& x_0 = \hat{x}(t_k), && && \text{(14c)}\\
& u_t \in \mathcal{U}, && t = 0, \dots, N-1, && \text{(14d)}\\
& d(x_t, \mathcal{P}_k) \geq r_{\min}, && t = 0, \dots, N, && \text{(14e)}
\end{aligned}$$

solved at every control instant t_k. Two properties of the way (14) meets the robot are worth stating before the design is read, because both are deviations from the textbook receding-horizon implementation.

The sampling time of the model is not the period of the loop. The step Δt in (14b) is the *prediction* step, 0.35 s in the deployed profile, while the program itself is re-solved at 8 Hz. The horizon is therefore re-planned about 2.8 times per predicted node, and the first predicted node already lies 0.35 s of open loop ahead of the state that produced it. The two quantities are independent design variables and are treated as such throughout.

Only u_0 would be applied, but it is not. The tracker does not publish $u_0^\star$. It walks the predicted trajectory $x_{0:N}^\star$ out to a fixed look-ahead distance and publishes the point it reaches as a geometric setpoint; a proportional law downstream converts that setpoint into the body twist actually sent to the locomotion layer, at 20 Hz. The optimizer is thus used as a *reference generator* rather than as a controller, and the loop closed on the robot is the cascade of that law with the locomotion policy rather than (14) itself. Section 5.1 bypasses the stage deliberately, so that the numerical results measure the optimization layer and not the tuning of the law beneath it.

Three features of (14) then determine the whole design:

- **The goal is not reachable within the horizon.** With $N\Delta t = 5.25$ s and a cruise speed of 0.27 m/s the horizon spans about a metre and a half, while the goal may be tens of metres away and outside the sensed region altogether. The cost (14a) therefore cannot be a distance to x_g; it must be a distance to a *reference trajectory* produced by a separate, cheaper program that looks further ahead. That program is the $A^\star$ search of Section 3.1.

- **The obstacle set is a point cloud, not a polytope.** Constraint (14e) is non-convex, and its description changes at every cycle. It is relaxed into a smooth penalty (Section 3.3.2), which keeps the NLP smooth and always feasible at the price of losing the hard safety guarantee.
- **The cost is parametrised by θ.** The eleven weights in θ are not physical quantities; they are design variables of a second-level optimization problem (Section 4.5).

3. Formulation of the optimization program

3.1. Reference generation: $A^\star$ as a discrete program

Section 2.6 established that the goal lies far outside the horizon, so the cost of (14a) must measure the distance to a reference trajectory rather than to x_g. That reference is produced once per replanning cycle by a discrete search on the occupancy grid of Section 2.4. Cells above the occupancy threshold are removed from the graph; the remaining 8-connected edges carry the cost

$$g(n \to n') \;=\; c_{\text{move}} \cdot \delta \cdot \left[1 + w_{\text{obs}} \left(\frac{P(n')}{P_{\text{thr}}}\right)^2\right], \qquad c_{\text{move}} \in \{1, \sqrt{2}\}, \tag{15}$$

and a standard $A^\star$ search returns the cheapest path between two cells. The normalised quadratic in (15) is what produces medial-axis-like paths: a cell just below the hard threshold costs $(1 + w_{\text{obs}})$ times an open cell, so the search prefers the middle of a corridor to a shorter path that grazes a wall, while a doorway is never closed outright. The Euclidean heuristic is admissible because the bracket is bounded below by 1, so no edge can be cheaper than its Euclidean length; the search is therefore an exact $A^\star$ and returns a cost-optimal path on the current occupancy grid.

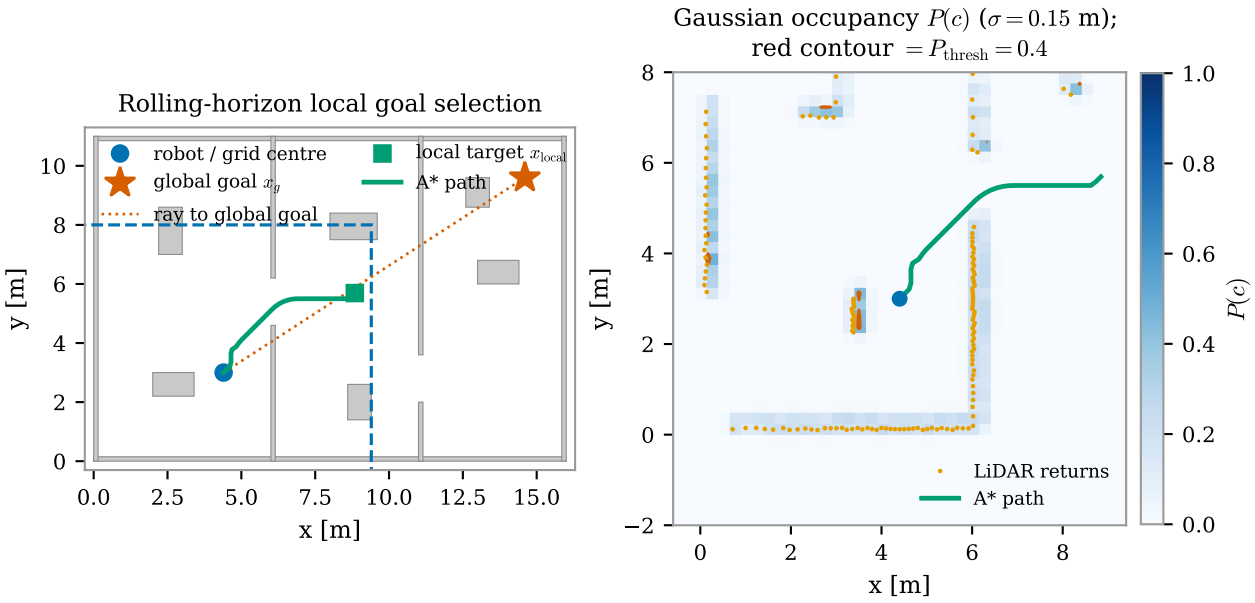

Figure 5: Rolling-horizon local goal selection (left) and the corresponding occupancy grid with the $A^\star$ path (right). The window $\mathcal{W}(\hat{x})$ is re-centred on the robot every replanning cycle rather than covering the whole map, so when the global goal x_g falls outside it a local target is projected onto the window boundary along the ray towards x_g and $A^\star$ plans only up to that point. This is what keeps the discrete search cheap regardless of how far the goal actually is, at the cost of the target rule discussed next.

What the discrete layer is for. Choosing *which side* of an obstacle to pass on is a discrete decision. Take a trajectory that passes on the left and one that passes on the right: turning the first into the second means moving it continuously through intermediate trajectories, and at least one of those must run through the obstacle. The feasible set is therefore disconnected, with one component per side, and a gradient-based solver stays in the component its initial guess falls into: it converges to the best trajectory on one side and never learns that the other side existed. Writing the choice into the NLP would require binary variables and turn it

into a mixed-integer nonlinear program, whose solution time is not compatible with a control loop at 8 Hz. A graph search represents the same choice natively, and on a 60×60 grid it costs a few milliseconds. Delegating the decision to the A* algorithm is therefore what leaves the continuous program smooth, with a box-shaped input set and no integer variables, the structure that Sections 4.1 and 4.2 exploit throughout.

The price to pay for such delegation is that the choice is made before the MPC is consulted and the MPC has no way to revise it: it can only deform the trajectory within the class it has been given. The two ways this breaks are opposite to each other and both are measured in Section 5. When the discrete layer *does not commit* (an obstacle squarely on the line to the goal, two routes of equal cost), the smooth program inherits two competing basins of its own, and small perturbations flip the solution from one to the other (Section 5.2). When it *commits on poor information*, the MPC follows it faithfully into a dead end, because questioning the reference is not its job (Section 5.6.1).

Choosing the target inside the window. The search of (15) runs on the local grid, and the global goal is normally outside it, so a target on the window has to be selected first. This is a selection over the free cells of the window boundary,

$$x_{\text{local}} \;=\; \underset{c \,\in\, \partial\mathcal{W}(\hat{x}) \cap \mathcal{F}_k}{\arg\min} \Big[\; d_k\big(c,\, x_g\big) \;+\; w_\tau\, \tau_k(c) \;\Big], \tag{16}$$

where $\mathcal{F}_k$ are the free boundary cells, d_k is a distance to the goal, and τ_k penalises cells the robot has already visited without making progress. Only boundary cells are candidates, because the boundary is where the direction of travel is decided; an interior cell would stop the robot half a window short of the edge for no reason. The substantive choice in (16) is the metric d_k. Two target rules were implemented and compared; only the second is an instance of (16) at all, and that is itself part of the story.

The first, which the stack shipped with, does not solve (16) at all. It takes the last free cell along the ray from the robot to the goal, which is a geometric shortcut rather than a minimisation,

$$x_{\text{local}} \;=\; \begin{cases} \text{cell}(x_g), & x_g \in \mathcal{W}(\hat{x}), \\ \text{ray}(\hat{x} \to x_g) \cap \partial\,\mathcal{W}(\hat{x}), & \text{otherwise}, \end{cases} \tag{17}$$

and the robot advances one grid width at a time towards the goal; if the projected cell is occupied, a breadth-first search returns the nearest free one. The rule is cheap, needs no map at all, and is memoryless and purely directional: whatever the map contains, the target lies on the line to the goal. In front of a concave obstacle that line enters the concavity, so the target does too.

It would be convenient if the defect stopped there, because then (16) with a Euclidean d_k would already repair it. It does not. Figure 6 shows the robot at the mouth of a horseshoe with the map it has actually observed: the ray projection selects a boundary cell 30.2 m of travel from the goal, and the arg min of $\|c - x_g\|_2$ over the same free boundary cells selects one 30.4 m away, a different cell, and just as wrong. Both are near the closed end, because in a straight line both are close to the goal. The straight-line distance rates a cell inside the concavity and one outside it almost equally, and where the two happen to tie exactly it separates them on numerical noise and reverses the choice at the next cycle. Section 5.6.1 shows that this is enough, on its own, to trap the robot.

The second instance is the deployed one. It keeps (16) and replaces d_k with the *geodesic* distance to the goal over the free cells of the accumulated map: a wavefront propagated from x_g, one scalar field per replanning cycle, with no path extracted from it. Space that has not been observed yet counts as free. This is the optimistic assumption of frontier exploration, and it is what makes the field correct itself as the sensor discovers: while the far end of a corridor is unseen the wavefront runs through it and entering is the right decision, and as soon as that end is observed the same cell becomes distant and the target is dropped. A candidate the wavefront cannot reach at all is discarded rather than scored with a Euclidean fallback, which would readmit the metric at fault; if no candidate is reachable on the known map the planner falls back to (17), which at least moves the robot towards unknown ground. The memory term $w_\tau \tau_k$ is implemented but carries zero weight in the deployed profile, for the reason measured in Section 5.6.2. In the same scene the geodesic arg min selects a cell 7.4 m from the goal, on the far side of the obstacle's arm, and $A^\star$ routes out of the concavity instead of further into it.

Commitment between replanning cycles. Two boundary cells lying on opposite sides of an obstacle can carry almost the same geodesic cost while belonging to different homotopy classes. The arg min then alternates between them from one cycle to the next, and the robot sways on the spot without ever committing. The deployed rule therefore retains the candidate that best continues the previous choice unless a competitor beats it by a margin of 2.0 m of geodesic cost, and clears that memory whenever the global goal changes, since a target inherited from a previous mission is simply the wrong direction. This is the ambiguity of Section 5.2

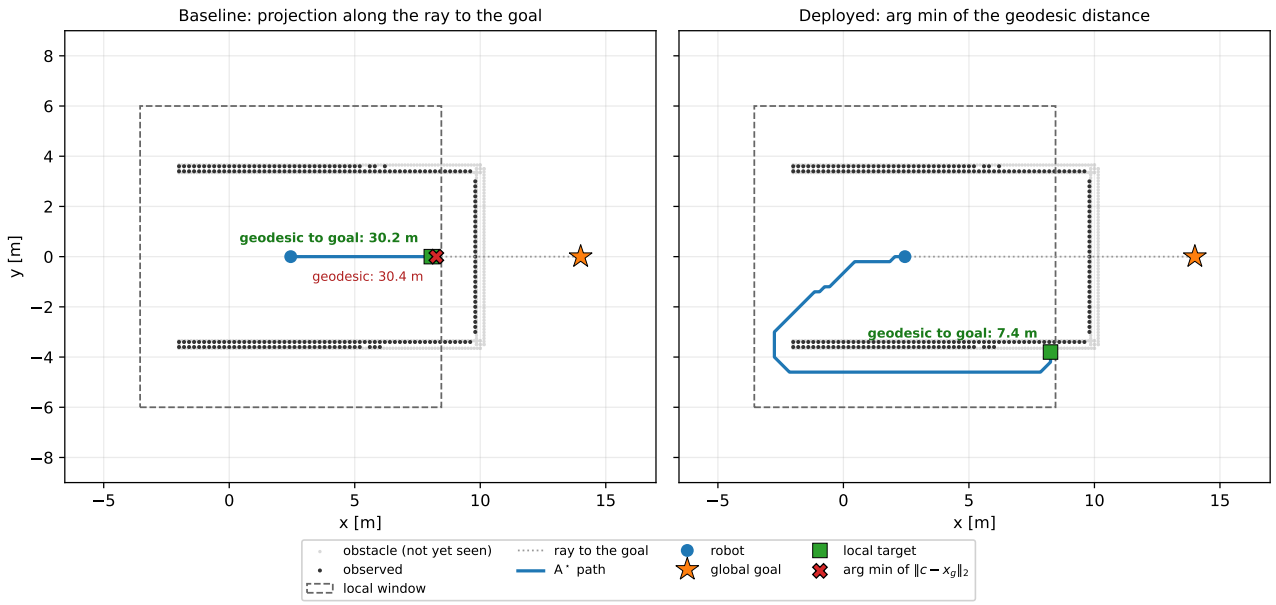

Figure 6: The same scene, the same window and the same observed map; only the metric of (16) changes. The robot sits at the mouth of a horseshoe and the map is the one it has actually accumulated by driving in, not the full world (light grey), which is what makes the failure reproducible. Left: the deployed projection (17) puts the target at the closed end, and the arg min of the Euclidean distance over the same free boundary cells lands beside it; the two rules differ, the outcome does not, because both are $\sim$30 m of travel from the goal. Right: with the geodesic d_k the target moves to the far side of the lower arm, 7.4 m away, and the $A^\star$ solution leaves the concavity. Drawn from the deployed planner and profile, not sketched.

seen one layer higher up: where the smooth program resolves it by breaking symmetry numerically, the discrete layer resolves it by declining to switch without evidence.

The geodesic field is the one expensive part of the planning layer, because the wavefront covers the whole known map rather than the window alone: it measures a median of 0.11 to 0.15 s per replanning cycle depending on the world, with a worst case of 0.27 s, which against the 1 Hz replanning rate is between a tenth and a quarter of the period and leaves the 8 Hz MPC untouched. The resulting grid path is finally resampled and smoothed with a moving average before it is handed to the MPC, to remove the zig-zag that the 8-connected discretisation introduces.

3.2. Model discretization

With sampling time Δt, the lag subsystem (12) is linear and time-invariant, so it is discretised *exactly* under a zero-order-hold input. Defining $\nu_i = 1 - e^{-\Delta t/\tau_i}$ for $i \in \{v_x, v_y, \omega\}$, the velocity update is the exact solution of (12) over one step,

$$v_{x,k+1} = (1-\nu_{v_x})v_{x,k} + \nu_{v_x} u_{1,k}, \qquad \omega_{k+1} = (1-\nu_\omega)\omega_k + \nu_\omega u_{3,k}, \tag{18}$$

and analogously for v_y with ν_{v_y}. No approximation is involved here and none should be claimed: this is the Lagrange formula for a linear time-invariant (LTI) system under a piecewise-constant input, and it is exact at the sampling instants. On the deployed G1 profile it is straightforward, since $\nu = 1$ reduces (18) to $v_{k+1} = u_k$; the statement is retained because it is the part of the model that would carry the weight on hardware, where τ is real.

The pose, by contrast, is genuinely nonlinear and must be integrated numerically. Two possibilieties are implemented and compared:

$$\text{explicit Euler:} \quad p_{k+1} = p_k + R(\psi_k)\, v_{k+1}\, \Delta t, \tag{19}$$

$$\text{mid-point (RK2):} \quad p_{k+1} = p_k + R\big(\psi_k + \tfrac{1}{2}\omega_{k+1}\Delta t\big)\, v_{k+1}\, \Delta t. \tag{20}$$

Both evaluate the rotation on the *post-lag* velocity v_{k+1}, a semi-implicit choice that costs nothing and removes the one-step delay that an explicit scheme would introduce between a command and the predicted displacement. The mid-point rule differs from Euler by one addition inside the argument of the rotation.

The results shown in the following compare two integrators outside the optimizer, which is only meaningful if the map assembled inside the NLP is the same one. That is checked rather than assumed: a single transition is extracted from the CasADi program built by the tracker, by reconstructing $f(x_0, u_0)$ from the first six equality constraints, and compared with the reference implementation of (19) and (20). The residual is zero to machine precision for both schemes.

Errors quantification according to the discretization method. The global error of each scheme was first fitted against the step size on three motion regimes, over a 2.80 s window, with the exact solution available in closed form: at constant body velocity the pose traces a circular arc. The measured orders are 1.00 and 2.00, exactly the first and second order the truncation analysis predicts, and the result is identical across regimes. The step grid is built by halving from the deployed Δt, so the fit begins at the step the controller actually uses rather than near it.
It is worth noting that a constant-velocity arc is not the regime in which the MPC works. The optimizer applies a *different* input at every node, and over a real horizon the commanded yaw rate changes sign.
The comparison is repeated on 12 horizons taken from a bag of data recorded while G1 is navigating in a mapless MuJoCo world and re-solved with the deployed profile, each integrated with its own optimal input sequence. The step is varied by subdividing the prediction interval while the input stays constant over each Δt, which keeps an order measurable on a real trajectory; the reference is the exact arc on the same post-lag velocity sequence, so the only difference between the three trajectories is how $R(\psi)$ is evaluated.

Table 2: Median position error at the end of the horizon, over 12 horizons re-solved from a bag of data recorded during the navigation, against the exact arc on the same velocity sequence. The first row is the deployed step; the others subdivide the prediction interval without changing the input.

Δt [s]	error, Euler [m]	error, mid-point [m]	ratio
0.3500	2.08×10^{-2}	3.66×10^{-4}	57
0.1750	1.04×10^{-2}	9.14×10^{-5}	113
0.0875	5.18×10^{-3}	2.28×10^{-5}	227
0.0437	2.59×10^{-3}	5.71×10^{-6}	453
0.0219	1.29×10^{-3}	1.43×10^{-6}	906

The orders survive the change of regime (1.00 and 2.00), but the error does not. At the deployed step the median is 2.08×10^{-2} m for Euler against 3.66×10^{-4} m for the mid-point rule, a factor 57 where the synthetic arc reports 114; the tails are 3.69×10^{-2} m and 7.16×10^{-4} m. The gap is the regime and not an inconsistency: over these horizons the commanded yaw rate spans 0.54 rad/s, so the per-step errors partially cancel each other out, while the arc accumulates them along a single turn. Measuring on the synthetic regime alone would have overestimated the benefit by a factor of two.

Where the floor is. One number bounds the discussion. Holding the velocity at its post-lag value across the interval (the semi-implicit choice made in (19) and (20)) neglects a displacement of 8.01×10^{-4} m per horizon, because the model's velocity rises towards the command instead of arriving at it. That is larger than the mid-point truncation error itself. Below that level refining the pose scheme doesn't improve the results: a different modelling choice already dominates and the second order is not the binding approximation.

Closed loop analysis. Prediction fidelity and executed cost are different quantities. Running the two schemes on the same worlds with everything else fixed moves the median closed-loop cost by at most 2.3%, and the sign changes from world to world, the signature of run-to-run variation rather than of an improvement.

Table 3: Closed-loop cost with the two integrators, same profile otherwise. A positive Δ means the mid-point rule is cheaper.

world	median cost, Euler	median cost, mid-point	Δ [%]
u_trap	163.5	167.2	-2.3
narrow_gap	110.7	108.2	2.3

The reason is structural and was stated in Section 2.6: only the first input is ever applied, and $A^\star$ replans at 1 Hz, so the accuracy of the far end of the prediction reaches the executed trajectory only indirectly. The

mid-point rule is deployed because it is a strictly better prediction for the price of one addition, not because it was measured to steer better.

3.3. The nonlinear MPC program

3.3.1 Decision variables, cost, and the parametrisation of the program

The NLP is written in the *simultaneous* form, also called direct multiple shooting: both the state and the input sequences are decision variables and the dynamics enter as equality constraints,

$$z \;=\; \big(X, U\big), \qquad X = [x_0, \ldots, x_N] \in \mathbb{R}^{6\times(N+1)}, \quad U = [u_0, \ldots, u_{N-1}] \in \mathbb{R}^{3\times N}. \tag{21}$$

With $e_k = x_k - x_{\mathrm{ref},k}$ the cost is

$$J(z;\theta) \;=\; \sum_{k=0}^{N-1} \Big[e_k^\top Q\, e_k \;+\; u_k^\top R\, u_k \;+\; J_{\mathrm{obs}}(x_k) \Big] \;+\; \sum_{k=1}^{N-1} R_{\mathrm{jerk}} \|u_k - u_{k-1}\|_2^2 \;+\; e_N^\top Q_T\, e_N + J_{\mathrm{obs}}(x_N), \tag{22}$$

with $Q = \mathrm{diag}(Q_x, Q_y, Q_\psi, 0, 0, 0)$, $Q_T = \rho_T\, Q$ and $R = \mathrm{diag}(R_{v_x}, R_{v_y}, R_\omega)$. Three properties of (22) deserve comment.

The stage weight is only semi-definite. No velocity state is penalised: Q has rank 3 out of 6. The velocities are shaped indirectly, through the lag dynamics and through the input penalty R. This is what keeps the reference generator simple, since an $A^\star$ path carries no velocity information, and the price is that the cost contains no direct damping of the velocity states.

The rate penalty couples consecutive inputs. The term $R_{\mathrm{jerk}}\|u_k - u_{k-1}\|_2^2$ is the discrete analogue of an input-rate penalty, and it is the only term that puts off-diagonal blocks between adjacent input stages in the Hessian. Its purpose is physical: a legged platform tracks a smooth velocity profile far better than a chattering one. Note that the sum starts at $k = 1$: the transition from the command applied on the previous cycle to u_0 is not penalised, so the first predicted move is free to jump. That is a deliberate omission (the program is re-solved at 8 Hz and would otherwise inherit a memory of its own past output), but it means the smoothness the term buys is smoothness *within* a horizon, not across consecutive ones.

The terminal cost is a scaled copy of the stage cost. $Q_T = \rho_T Q$ is a heuristic, neither the solution of a Riccati equation nor an exact cost-to-go, and like Q it acts only on the three position and heading components. ρ_T is therefore a tuning parameter and is treated as one, appearing among the eleven weights of θ in Section 4.5. (The gap between this heuristic and the true infinite-horizon cost-to-go of the linearised dynamics is not quantified in this report; doing so would require the Riccati comparison, which the semi-definite Q would first oblige us to regularise.) The stability argument is not carried by Q_T at all; what would carry it, and why the deployed profile does without it, is the subject of Section 3.3.5.

Comparison between multiple and single shooting. The standard argument for multiple shooting is that it keeps the Jacobian sparse and banded, where a condensed single-shooting formulation, in which states are eliminated by recursive substitution and so the inputs are the only variables, would produce a small but dense problem. A direct comparison between the two approaches can be performed, with the aim to show why a multiple-shooting approach was chosen.

Table 4: Multiple (M) against single (S) shooting on the same instance, at the deployed profile. Dimensions, densities and iteration counts are exact; each timing is the fastest of three cold solves, with no warm start. The table shows the comparison for the number of variables, number of constraints, the Hessian density, the number of iterations and the solve time needed by each method for different lenghts of the prediction horizon N.

N	vars M / S	cons M / S	hess dens. [%] M / S	iter M / S	solve [ms] M / S
5	51 / 15	56 / 20	12.11/100.00	19 / 18	71 / 64
10	96 / 30	106 / 40	6.51/100.00	29 / 21	137 / 135
15	141 / 45	156 / 60	4.45/100.00	35 / 24	188 / 218
25	231 / 75	256 / 100	2.73/100.00	39 / 31	299 / 524
40	366 / 120	406 / 160	1.72/100.00	39 / 39	393 / 961
60	546 / 180	606 / 240	1.16/100.00	39 / 29	593 / 2106

The structural half of the claim holds exactly, and the Hessian is where it shows: at $N = 15$ the program shrinks from 141 variables and 156 constraints to 45 and 60, while the Hessian density goes from 4.45% to 100.00%, a full matrix. Condensing does trade a large banded problem for a small dense one.
If the last two columns together are analyzed, the reason why employing a multiple-shooting approach is beneficial becomes clear. At $N = 60$ the condensed form converges in *fewer* iterations than the sparse one, 29 against 39, and still takes three and a half times as long. The cost is therefore not in the number of Newton steps but in what each one costs, which is a factorisation of the Hessian, that as shown in the middle column goes from a few per cent dense to a full dense matrix.
Two caveats keep this from being a general result. First, at $N = 15$ and $N = 40$ the two parametrisations converge to *different* minima, 6.7% and 5.8% apart in cost. That is not an implementation error (the program is non-convex and two parametrisations follow different optimization paths, so they can land in different basins), but it does mean that on those two rows, one of which is the deployed horizon, the timings compare solves that did not solve the same problem. Second, the conditioning argument usually made for multiple shooting does not show up here at all: single shooting integrates the model in *open loop* over the whole horizon, so its error compounds step by step and the problem degrades with N and with any instability of the plant, but on a kinematic and stable model that defect never surfaces. This comparison therefore supports the choice made here; it is not evidence for the general case.

3.3.2 Obstacle avoidance as a smooth penalty

The clearance constraint (14e) is replaced, in the deployed configuration, by a penalty evaluated on the $M = 8$ nearest LiDAR returns $\{p_j\}$ inside a search radius of 2.0 m:

$$J_{\text{obs}}(x_k) \;=\; W_{\text{obs}} \sum_{j=1}^{M} \left[\underbrace{\tfrac{1}{2}\Big(1 - \tanh\big(\tfrac{\alpha_{\text{obs}}}{2}(d_{k,j} - r_{\text{obs}})\big)\Big)}_{\text{sigmoid zone}} \;+\; \underbrace{2\max\big(0,\; r_{\text{obs}} - d_{k,j}\big)^2}_{\text{penetration penalty}} \right], \tag{23}$$

with $d_{k,j} = \sqrt{(p_{x,k} - p_{j,x})^2 + (p_{y,k} - p_{j,y})^2 + \varepsilon_d}$ and $\varepsilon_d = 10^{-6}$ making the distance differentiable at zero, where the Euclidean norm is not. The sigmoid is written with tanh rather than as $1/(1 + e^{\alpha_{\text{obs}}(d - r_{\text{obs}})})$: the two are algebraically identical, but the logistic form overflows for large arguments while tanh saturates cleanly, and the argument does get large, since the sentinel padding described below puts absent returns at 10^3 m, which at $\alpha_{\text{obs}} = 6.00$ makes the exponent $\approx 6 \times 10^3$.
The hybrid structure is deliberate, and Fig. 7 is the argument for it. The sigmoid saturates at W_{obs} as $d \to 0$: on its own it is a *bounded* penalty, so a sufficiently strong tracking term can always buy its way through an obstacle. Worse for the solver, its gradient decays on both sides of r_{obs}, so the restoring force fades exactly where the robot most needs pushing out. At the deployed $\alpha_{\text{obs}} = 6.00$ the decay is mild (at two centimetres of clearance the sigmoid still retains about a third of the restoring force it has at $d = r_{\text{obs}}$), but it is governed by α_{obs} and it sharpens fast: at four times that steepness the same point retains essentially none. Steepening the barrier to improve avoidance is therefore precisely what destroys its gradient where avoidance has already failed. The quadratic term removes both defects inside the safety radius: it is unbounded, and its gradient grows linearly with penetration regardless of α_{obs}.
The price is the third panel. The curvature of (23) scales as $\alpha_{\text{obs}}^2 W_{\text{obs}}$, so a steep, heavy penalty injects large and rapidly varying eigenvalues into the Hessian precisely where the solution lives. The deployed tuning sits at the crossover: the peak curvature contributed by the sigmoid, $\approx 0.096\,\alpha_{\text{obs}}^2 W_{\text{obs}}$, is just below the $4W_{\text{obs}}$ contributed by the quadratic term, so neither dominates, and doubling α_{obs} already puts the sigmoid a factor of three above it, with the conditioning of the NLP following.
Relaxing a hard constraint into a penalty has three consequences worth stating explicitly. *(i)* The NLP is always feasible: no state constraint can render the problem unsolvable when the robot is already too close to an obstacle, which for a real-time loop with no fallback trajectory is a genuine robustness property. *(ii)* Safety is no longer guaranteed; (23) is not an exact penalty, so a finite W_{obs} admits a finite violation, and Section 3.3.3 makes that statement quantitative. A control-barrier formulation [1] would recover the guarantee, at the price of feasibility and of a harder NLP. *(iii)* The penalty is evaluated only at the $N + 1$ discrete nodes, so nothing forbids the continuous inter-sample trajectory from clipping a thin obstacle.
Point *(iii)* is worth checking rather than leaving as a caveat, because it is a one-line computation. Between two consecutive nodes the robot advances at most $v_{x,\max}\Delta t$, against a keep-out radius r_{obs}, so the penalty is meaningful only while

$$\frac{v_{x,\max}\,\Delta t}{r_{\text{obs}}} \;<\; 1. \tag{24}$$

On the deployed profile the numerator is 0.40×0.35, i.e. about 14 cm against 0.40 m of radius, so the ratio is 0.35: safe, but a third larger than before the input envelope was widened. It is reported rather than assumed because

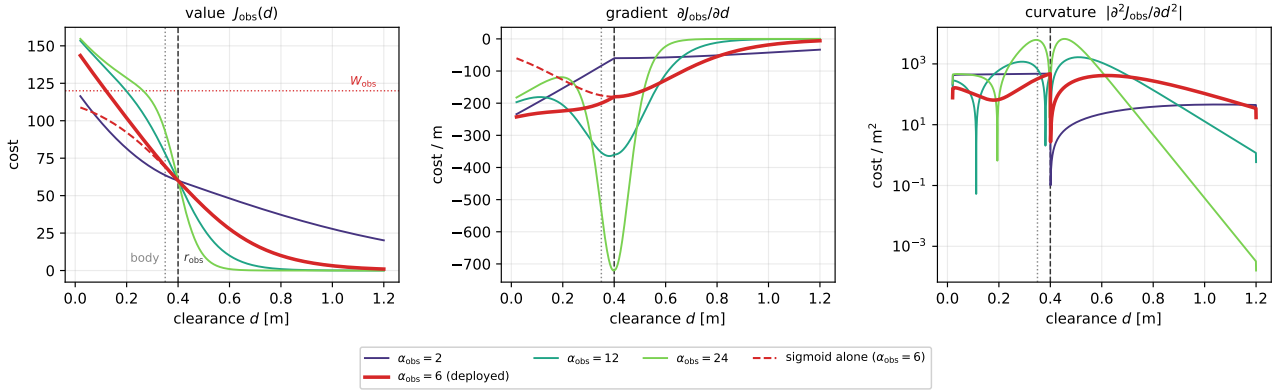

Figure 7: The hybrid penalty (23) against clearance, for four steepness values with the deployed $\alpha_{\text{obs}} = 6.00$ in bold ($W_{\text{obs}} = 120$, $r_{\text{obs}} = 0.40$ m; the dotted line marks the body radius). Dashed: the sigmoid zone alone at the deployed steepness, which saturates at W_{obs} and whose gradient fades inside the obstacle, the two defects the quadratic term repairs. Right, on a logarithmic scale: the curvature, which grows as α_{obs}^2 and is what degrades the conditioning of the NLP; the notches are sign changes. Evaluated through the same function the tracker builds into the NLP, not redrawn from the formula.

it is not invariant: it couples two parameters tuned for unrelated reasons (the sampling time of Section 5.3 and the input envelope of Table 1), and lengthening Δt to buy horizon walks it towards one. Above one the obstacle term is absent between nodes, and geometric avoidance falls entirely to the $A^\star$ layer on the inflated grid.

Sentinel padding. M is fixed. When fewer than M returns lie inside the search radius, the missing rows of the obstacle parameter are filled with a sentinel at 10^3 m, whose contribution to (23) is numerically zero. This apparently cosmetic detail is what allows the NLP to be built *once*: the number of terms, and therefore the sparsity pattern of the Jacobian and Hessian, never changes between cycles, so the symbolic factorisation and the warm start remain valid regardless of how cluttered the scene is.

3.3.3 The obstacle as a true constraint: exact ℓ^1 penalty

The alternative to a penalty in the cost is a constraint with a slack. The deployed code implements both, selectable by a parameter: the obstacle becomes

$$d(x_k, p_j) \;\geq\; d_{\text{safe}} - s_{jk}, \qquad s_{jk} \geq 0, \qquad k = 1, \ldots, N, \tag{25}$$

with the slack penalised in the cost either linearly ($\rho_s \sum s$) or quadratically ($\rho_s \sum s^2$). The question this settles is the one Section 3.3.2 could only raise: at what weight, if any, does the relaxation stop being a relaxation, so when the soft constraint becomes a hard one.

Penalising the slack in ℓ^1 makes the penalty *exact*: above a finite threshold ($\mu^\star$, the norm of the multipliers of (25) at the solution), the penalised and the constrained problem share their minimisers and the slack vanishes identically, so beyond that weight the relaxation is no longer one. On the deployed problem the slack is already identically zero at $\rho_s = 1 \times 10^5$. Penalising in ℓ^2 never gets there at all: the slack decays as $1/\rho_s$ and the constraint is violated, however slightly, at every weight.

That answer is also the reason the obstacle is left in the cost. A constraint that is genuinely hard can be genuinely infeasible, and it becomes so exactly when the robot is already inside d_{safe}, the one situation in which a loop with no fallback trajectory must still return a command. This is consequence *(i)* of Section 3.3.2 read in the other direction: what the penalty gives up in guarantees it buys back in always having an answer. Below the threshold nothing is gained either, since the constraint is soft again and a soft constraint carrying one slack per node and per obstacle is (23) with more variables.

What it would cost to deploy. Switching the obstacle from the cost to the constraints takes the inequalities from 60 to 300 and the iteration count up by a factor of several. The deployed iteration cap would not suffice. This is also what makes the solver comparison of Section 4.2.3 possible at all: the same system can be placed in two regimes that differ by a factor five in the number of inequalities, which is exactly the axis along which the standard advice on solver choice is stated.

3.3.4 Constraints

The deployed program carries only equality constraints and input bounds:

$$x_{k+1} - f(x_k, u_k) = 0, \qquad k = 0, \dots, N-1, \tag{26a}$$
$$x_0 - \hat{x} = 0, \tag{26b}$$
$$v_{x,\min} \le u_{1,k} \le v_{x,\max}, \qquad k = 0, \dots, N-1, \tag{26c}$$
$$|u_{2,k}| \le v_{y,\max}, \qquad |u_{3,k}| \le \omega_{\max}, \qquad k = 0, \dots, N-1, \tag{26d}$$

with $v_{x,\min} = -0.20$ m/s, $v_{x,\max} = 0.40$ m/s, $v_{y,\max} = 0.20$ m/s and $\omega_{\max} = 0.40$ rad/s (Table 1). Three remarks:

- **No state constraints.** Neither the obstacle clearance nor the velocity states are constrained; the former is penalised, the latter are bounded implicitly because (18) is a convex combination of a bounded command and the previous velocity, so $v_{x,k}$ stays inside the input box for all k whenever $v_{x,0}$ is, which the state estimate does not guarantee, although on the deployed profile the degenerate lag of Section 3.2 collapses (18) to $v_{k+1} = u_k$ and the question does not survive the first node. The feasible set of the NLP is therefore a box intersected with an affine-in-X manifold, and a feasible point is always trivially available (integrate any admissible input sequence).
- **The input box is asymmetric.** Constraint (26c) admits reversing, but at half the forward speed, and (26d) caps strafing at the same fraction. Neither asymmetry is a safety derating applied to a symmetric machine: the lateral one is kinematic, since a biped strafes genuinely more slowly than it walks forward, while the backward one keeps the optimizer from adopting a retreat as a cheap shortcut at every stop. It is not free, however (pricing the retreat above the wide forward arc is one of the three structural facts behind the livelock of Section 5.6.1), and on hardware the bound may be tightened further to $v_{x,\min} = 0$, since the LiDAR sees poorly behind the robot.
- **Bounds are parameters, not constants.** The three velocity limits enter the NLP as CasADi parameters, so the supervisor can shrink them at runtime (Section 4.4) without rebuilding the problem, to slow the robot down in a crowded area, for instance, or to open the envelope again in free space.

3.3.5 Terminal ingredients and recursive feasibility

The deployed formulation carries no terminal set and no terminal constraint, so none of the standard guarantees of nominal stability and recursive feasibility [3, 8] applies. Recursive feasibility is nevertheless *structurally* present, for the reason given in Section 3.3.4: with no state constraints, every input sequence in the box is admissible, so the NLP always has a solution and the closed loop can never be aborted by infeasibility. What is lost is the guarantee that the closed loop converges: the tracking term can decrease along a trajectory that walks into a well of the obstacle penalty, and nothing but that penalty stops it.

It is worth saying plainly what that gap is. The controller plans 5.25 s ahead, executes the first step of the plan, and then starts again from scratch. Nothing ties one plan to the next, so a sequence of individually optimal plans need not go anywhere: each is the best available move inside a window that never looked past its own edge. The standard device for closing the gap is a condition on the last predicted state, end every plan in a configuration that can be held indefinitely; because the previous plan, shifted forward by one step, is then always still executable. For a walking robot that configuration is standing still, balance being the business of the policy of Section 2.3.2, so the condition reads $v(N) = 0$: there must exist a braking trajectory inside the horizon.

On the plant used in this report that condition holds without being imposed. Section 2.5 sets $\tau_{v_x} = \tau_{v_y} = \tau_\omega = 0.00100$ s against $\Delta t = 0.35$ s, so every discrete lag coefficient is 1.000000 and the velocity update collapses to $v_{k+1} = u_k$: the model reaches zero velocity in a single sampling step, from any speed. A braking trajectory therefore always exists and the guarantee comes for free, for a reason that belongs to the simulation and not to the robot.

Supplying it explicitly would cost very little. The constraint is imposed softly, as

$$v_i(N) \le s_i, \qquad -v_i(N) \le s_i, \qquad s_i \ge 0, \qquad i \in \{x, y, \omega\}, \tag{27}$$

with $\rho_s \sum_i s_i$ added to the cost: three slack variables and nine inequalities, against the 141 variables and 156 constraints the program already carries, and all of them added once rather than once per node. Penalising in ℓ^1 is what makes the soft form acceptable; by the result of Section 3.3.3 the constraint is effectively hard wherever it is satisfiable and yields instead of aborting the loop where it is not. The deployed code implements it, selectable by a parameter; the deployed profile leaves it off.

Measured with that parameter forced on, across cycles replayed from a recorded mission, the terminal slack never leaves zero (maximum 0, always admissible: yes), which is what the degeneracy above predicts, and the

optimal objective rises by between 2.2 and 7.0. Those same cycles carry objectives ranging from 6.6 to 567.6, so the identical increase reads as anything between 1.2% and 40.4% depending on which cycle is quoted: the relative figure describes its own denominator rather than the constraint. The absolute cost is small, roughly constant, and easy to account for; the plan has to predict its own braking, and the braking is discarded at the next cycle anyway.

Where the constraint would earn its place is on an actuator that is not instantaneous. Repeating a cycle with τ raised to 2 s, so that the lag coefficient falls to 0.16, produces the only strictly positive terminal slack in this study: the horizon is no longer long enough to stop. On hardware, where τ is fixed by the locomotion policy and the gait rather than chosen, $v(N) = 0$ inside a fixed horizon is precisely the condition that ties the cruise speed to the horizon length. This is the job currently done reactively, and outside the optimizer, by the adaptive velocity ceiling of Section 4.4.

3.3.6 On the absence of integral action

The formulation has no integral action and no disturbance estimator: the state (11) is not augmented with an input-disturbance model, and the reference is tracked in a purely proportional–predictive sense. For a nominal plant this is immaterial, but any constant unmodelled input (a mis-identified τ, a lateral drift of the gait, a slope) produces a persistent offset from the path rather than being integrated away. The offset is therefore a property of the formulation and not of the tuning, and no choice of Q removes it. Section **??** discusses the standard remedy, state augmentation with an integrated tracking error as in the velocity-form MPC of [11].

4. Optimization procedure

4.1. Problem dimensions and structure

Table 5 reports the size of the NLP at the deployed setting $N = 15$, $M = 8$. It is a medium-scale, sparse, smooth, non-convex problem: 141 decision variables, 96 equality constraints, 60 simple input bounds, and $2M(N + 1) = 256$ barrier terms in the objective; there are no obstacle *constraints*, so there is no obstacle multiplier and no non-trivial active set to identify: the entire inequality budget of the deployed program is the 60 box bounds on the inputs. A further 121 quantities enter as CasADi parameters: the measured state, the reference, the sentinel-padded obstacle positions and the three velocity bounds.

Table 5: NLP dimensions at the deployed configuration $N = 15$, $\Delta t = 0.35$ s, $M = 8$.

Quantity	Expression	Value
State variables	$6(N + 1)$	96
Input variables	$3N$	45
Total decision variables	$6(N + 1) + 3N$	**141**
Dynamic equalities	$6N$	90
Initial-state equalities	6	6
Total equality constraints	$6(N + 1)$	**96**
Input bounds	$4N$	**60**
Barrier terms (sigmoid + quadratic)	$2M(N + 1)$	256

The Jacobian of (26a) is block bi-diagonal, with 6×9 blocks coupling (x_k, u_k) to x_{k+1}; the Hessian of the Lagrangian is block tri-diagonal, the off-diagonal input blocks coming from the rate penalty. Nothing in the problem couples distant stages, so the linear-algebra cost of one interior-point iteration is linear in N; at the deployed size the constraint Jacobian is 1.73% dense and the Hessian 4.45%, and Table 6 shows both densities falling as N grows while the nonzero counts grow linearly.

4.2. Solver methodology

4.2.1 Overview

The NLP is solved with IPOPT [15], a primal–dual interior-point method with a filter line search. Writing the problem as $\min_z \phi(z)$ subject to $c(z) = 0$ and $z^L \le z \le z^U$, the bounds are replaced by a logarithmic barrier

$$\varphi_{\mu_b}(z) \;=\; \phi(z) \;-\; \mu_b \sum_i \Big[\ln\big(z_i - z_i^L\big) + \ln\big(z_i^U - z_i\big) \Big], \tag{28}$$

Table 6: Size and sparsity of the NLP against the prediction horizon, from the deployed profile. Bold: the deployed configuration.

N	vars	eq.	bounds	params	jac nnz	jac dens. [%]	hess nnz	hess dens. [%]
10	96	66	40	91	256	2.52	600	6.51
15	**141**	**96**	**60**	**121**	**381**	**1.73**	**885**	**4.45**
25	231	156	100	181	631	1.07	1455	2.73
50	456	306	200	331	1256	0.54	2880	1.39

the resulting equality-constrained problem is solved approximately by a Newton step on the perturbed Karush–Kuhn–Tucker (KKT) system

$$\begin{bmatrix} \nabla^2_{zz}\mathcal{L} + \Sigma & \nabla c(z)^\top \\ \nabla c(z) & 0 \end{bmatrix} \begin{bmatrix} \Delta z \\ \Delta\lambda \end{bmatrix} = - \begin{bmatrix} \nabla\varphi_{\mu_b} \\ c(z) \end{bmatrix}, \tag{29}$$

and μ_b is driven to zero. Each iteration requires one factorisation of the sparse symmetric indefinite matrix in (29), performed by MUMPS on the pattern of Section 4.1. The Newton step therefore needs exact first and second derivatives at every iteration: the gradient of ϕ, the Jacobian ∇c and the Hessian of the Lagrangian $\nabla^2_{zz}\mathcal{L}$. None of them is coded by hand or approximated numerically. All three are produced by CasADi through algorithmic differentiation (AD), which walks the expression graph of Section 4.1 and composes the derivatives of the elementary operations exactly, to machine precision. Section 4.2.2 measures what this costs and what the alternatives would cost; Section 4.2.3 asks whether the interior-point method assumed here is the right family of solver for this problem in the first place.

There is a pleasing coincidence between (28) and (23) that is worth naming: the obstacle term of the deployed program is itself a barrier, and α_{obs} plays the role of an inverse barrier parameter. Steeper means better avoidance and worse conditioning, which is exactly the trade-off μ_b governs inside the solver. The difference is that IPOPT drives its own barrier parameter to zero along a central path, whereas α_{obs} is fixed by the tuning and so in that case the barrier never relaxes.

4.2.2 Derivative computation

Every quantity the solver needs, like the gradient of the objective, the Jacobian of the constraints, the Hessian of the Lagrangian, is generated symbolically by CasADi and evaluated by generated code. The alternative is to employ other differentiation methods, such as finite differences. A comparison is done between the two approaches in order to assess if the second-order method above is affordable, computationally speaking.

Table 7: Accuracy of the finite-difference gradient against the step size, measured against the AD gradient. The optimum sits at $h = 1.49\times10^{-8}$ forward (theory: $\sqrt{\varepsilon} = 1.49\times10^{-8}$) and $h = 6.06\times10^{-6}$ central (theory: $\varepsilon^{1/3} = 6.06\times10^{-6}$).

step h	rel. error, one-sided	rel. error, central
1.00×10^{-4}	9.18×10^{-4}	2.15×10^{-8}
6.06×10^{-6}	5.56×10^{-5}	9.84×10^{-11}
1.00×10^{-6}	9.17×10^{-6}	5.29×10^{-10}
1.49×10^{-8}	1.27×10^{-7}	3.77×10^{-8}
1.00×10^{-10}	9.66×10^{-6}	4.36×10^{-6}

At a representative solve the objective has 141 variables. One evaluation costs 132.4 μs and one full gradient by reverse-mode algorithmic differentiation (AD) 193.7 μs, i.e. 1.46 objective evaluations; it is small and constant, the important point is that it doesn't grow with the number of variables. The same gradient by finite differences costs 142 evaluations for the forward case and 282 for the central one, both growing linearly with n and is less accurate at every step size: the best relative error attainable is 1.27×10^{-7} in the first case and 9.84×10^{-11} in the second one, against machine precision for AD. The measured optimal steps coincide with the theoretical $\sqrt{\varepsilon_M}$ and $\varepsilon_M^{1/3}$, ε_M being the machine epsilon.

For a real-time loop the cost settles the question: central differences alone would consume 30% of the 125 ms cycle budget, for a gradient that is worse, and the fraction grows linearly with the horizon. The ratio itself is a micro-benchmark on $\sim$100 μs timings and its second digit is not meaningful; what is stable, and is the point, is that it remains a small constant.

Exact Hessian against BFGS. Because AD supplies exact second derivatives at a cost comparable to the first, the full Newton system can be used rather than a quasi-Newton approximation.

Table 8: Interior-point iterations with the exact Hessian and with the limited-memory quasi-Newton approximation (L-BFGS), on the same instance. Bold: the lower iteration count.

Hessian	iterations	solve [ms]	$J^\star$	status
exact Hessian	**35**	328.4	3646	`Solve_Succeeded`
L-BFGS	89	1078.0	3646	`Solve_Succeeded`

On the same instance the exact Hessian converges in 35 iterations against 89 for the limited-memory quasi-Newton approximation, which means a 61% reduction, and both reach the same objective value. The total time is closer than the iteration counts suggest, because each quasi-Newton iteration is cheaper: this is the Newton / quasi-Newton trade-off. Section 4.2.3 shows the other side of the same coin, where the exact Hessian of a non-convex program produces an indefinite quadratic program (QP) subproblem.

4.2.3 Interior point and active set comparison

The standard advice is that active-set methods win when the inequalities are few and interior-point methods win when they are many. This stack can be moved between the two regimes without changing anything else, because the obstacle formulation is switchable: with the obstacles in the cost the program carries 60 inequalities, and with the obstacles as genuine constraints (Section 3.3.3) it carries 300. That makes the advice testable rather than quotable.

Table 9: The same three instances solved by a primal–dual interior-point method and by an SQP with an active-set QP solver, in both obstacle regimes: *penalty* keeps the obstacles in the cost, ℓ^1 carries them as constraints with slacks (Section 3.3.3). Both solvers are started cold. Bold: the faster of the two. *fail* marks a run in which the active-set QP subproblem could not be solved at all. †the only large-regime run that returns a solution ends at an objective 0.2% away from IPOPT's, so that ratio is indicative rather than an exact like-for-like comparison.

regime	ineq.	IPOPT ms / iter	SQP ms / iter	speed-up	same min.
penalty (obstacles in the cost)	60	**67** / 20	146 / 6	2.2×	yes
penalty (obstacles in the cost)	60	**89** / 23	860 / -1	9.6×	no
penalty (obstacles in the cost)	60	**225** / 77	275 / 7	1.2×	yes
ℓ^1 (obstacles as constraints)	300	**146** / 35	9307 / 16	63.6×	no
ℓ^1 (obstacles as constraints)	300	**173** / 46	13204 / -1	76.4×	no
ℓ^1 (obstacles as constraints)	300	**162** / 45	11062 / -1	68.2×	no

The rule does not reproduce: the active-set solver does not win even in the small regime, where the interior-point method is already between 1.2× and 2.2× faster on the two instances both methods solve. The *direction* is confirmed, indeed the margin widens to 68.2× when the inequalities multiply; so the break-even point, if it exists, lies below the smallest regime into which this problem can be put.

The failures matter more than the margins. In three of the six runs the active-set QP subproblem could not be solved at all, and the SQP returned nothing after 0.9 to 13 s, two of those three in the large regime, where it fails on two instances out of three. So the comparison is not that one method is slower than the other: on this program the interior-point method returns a certified solution every time, and the active-set route returns one in half the cases.

Two cautions, without which the numbers would be misleading. First, the real advantage of an active set in MPC is warm starting *between consecutive solves*: the active set changes by a few rows per cycle and the factorisation is reused, which is the online active-set strategy the method was designed for. Both solvers are started cold here, deliberately, so as to favour neither, which removes from the active-set method precisely what makes it competitive in a receding-horizon loop. Second, the advice is stated for convex programs, and this one is not.

4.3. Optimality conditions on the deployed problem

Everything above assumes that what the solver returns is a solution. That is a verifiable statement, and this section verifies it on cycles extracted from a recorded mission rather than on a synthetic instance.

Table 10: Optimality diagnostics at sampled cycles of the recorded run `industrial_v6`. "active" counts equalities and active bounds together; "rank" is the rank of the Jacobian of the active constraints, so LICQ holds when the two coincide.

cycle	t [s]	active	of which bounds	rank	LICQ	cone dim.	$\lambda_{\min}$ proj.
0	0	103	7	103	yes	38	1.33×10^{0}
86	46	101	5	101	yes	40	1.26×10^{0}
173	85	98	2	98	yes	43	1.25×10^{0}
260	120	101	5	101	yes	40	1.31×10^{0}
347	153	97	1	97	yes	44	1.24×10^{0}
433	191	100	4	100	yes	41	1.26×10^{0}
520	245	100	4	100	yes	41	1.25×10^{0}
607	315	101	5	101	yes	40	1.28×10^{0}
694	358	98	2	98	yes	43	1.23×10^{0}

At 9 sampled cycles, the linear independence constraint qualification (LICQ) holds at every one (yes), the rank of the Jacobian of the active constraints equals their number; so the KKT multipliers exist and are *unique*. Strict complementarity holds at every one (yes), which means the critical cone coincides with the null space of the active Jacobian and the second-order condition can be checked exactly rather than conservatively. And the reduced Hessian is positive definite on that cone at every cycle (yes), the smallest projected eigenvalue over the profile being 1.23×10^{0}. Taken together these are a certificate of strict local optimality, which is the most that can be claimed for a non-convex program.

The result worth reporting is the shape of the active set. The critical cone holds between 38 and 44 dimensions across the sampled cycles and does not collapse: with 141 variables and 98 active constraints at the end, the freedom the controller has when it arrives is the freedom it had when it started. Almost every active row is an equality of the dynamics; the input bounds contribute a handful per cycle and never more. The controller is cost-driven for the whole mission.

That is not how it started. On a narrower envelope (lateral speed an order of magnitude below forward travel, reversing forbidden) the same diagnostic showed the cone collapsing to 8 dimensions, with the lateral and yaw limits saturated on essentially every step of the horizon: the controller was undergoing the trajectory rather than choosing it. Widening the envelope to $|v_y| \le 0.20$ m/s and $v_x \in [-0.20, 0.40]$ m/s was the recommendation that followed, and the table above is the run that acted on it (Section **??**).

4.4. Implementation and configuration

The program is built once, symbolically, with the CasADi `Opti` interface, and every quantity that changes between cycles is a *parameter* rather than a constant:

- $\hat{x} \in \mathbb{R}^6$, the measured initial state;
- $X_{\text{ref}} \in \mathbb{R}^{6\times(N+1)}$, the reference tracked by the Q block of (22);
- $\mathcal{O} \in \mathbb{R}^{2\times M}$, the (sentinel-padded) obstacle positions;
- the three velocity bounds of (26c)–(26d).

At the deployed $N = 15$ and $M = 8$ these account for $6 + 96 + 16 + 3 = 121$ parameter slots. Consequently the expression graph, the derivative code and the sparsity patterns are generated exactly once at start-up; a cycle only writes numbers into parameter slots and calls the solver. The graph is flattened with `expand: true`, which converts the `MX` graph into scalar `SX` operations and removes the interpreter overhead that would otherwise dominate a problem whose objective and constraints together expand to 3603 scalar operations. The first solve after start-up therefore pays a one-off construction and code-generation cost, 0.48 s at the deployed configuration and up to 0.78 s at the longest horizon tested (Section 5.3); this is the single deadline violation of the entire run, and the deployment absorbs it by holding the robot still during initialisation.

Solver options are deliberately austere: `max_iter`= 100, `print_level`= 0, `warm_start_init_point` enabled. No CPU-time limit is imposed inside IPOPT; the real-time protection lives in the supervisor instead, which is the more robust arrangement because it keeps the solver deterministic. A second reason is specific to the

architecture of Section 2.1: a non-converged iterate is still usable as a reference by the downstream setpoint stage, which would not hold if $u_0^\star$ were applied to the joints directly. The supervisor implements three mechanisms, all part of the deployed configuration and all of which exist because a loop at 125 ms per cycle cannot afford to wait for a perfect solution:

1. **Warm-start hygiene**, the cache is cleared on solver failure and on a cost spike (Section 4.2.3).
2. **Zero-velocity fallback**, after three consecutive IPOPT failures the tracker returns a zero input, which is always feasible and always safe; this is preferable to applying the last usable solution, which is by then stale. The fallback is deliberately transient: it skips a single cycle and then resets its own counter, because a latching fallback would leave a robot that never recovers.
3. **Adaptive input bounds**, when the observed failure rate exceeds 30% the supervisor shrinks $v_{x,\max}$ by 10% through the parameter interface, reducing the distance the horizon covers and thereby the number of active barrier terms, and walks it back up in 5% steps once the failure rate drops below 5%.

Unlike the mechanisms of a system that never exercises them, these do fire. Over the 8688 solves of the extended horizon campaign the solver failed 308 times, and the failures are strongly graded by horizon length: 0.7% at $N = 15$, 3.5% at $N = 26$ and 6.5% at $N = 40$. This is worth stating plainly, because it is the honest form of the argument for the deployed horizon: the long horizons are not merely slower, they are the ones on which a bounded iteration count stops being enough, and the fallback rather than the optimum decides the command.

4.5. Offline weight tuning by Bayesian optimization

The cost (22) and the obstacle term (23) are parametrised by eleven weights,

$$\theta = \big(Q_x,\, Q_y,\, Q_\psi,\, \rho_T,\, R_{v_x},\, R_{v_y},\, R_\omega,\, R_{\text{jerk}},\, W_{\text{obs}},\, \alpha_{\text{obs}},\, r_{\text{obs}}\big) \in \Theta \subset \mathbb{R}^{11}, \tag{30}$$

which are not physical quantities but the decision variables of a second-level problem

$$\hat\theta \;=\; \arg\min_{\theta\in\Theta}\; \mathcal{J}(\theta), \tag{31}$$

where $\mathcal{J}$ measures closed-loop navigation quality. This is the third optimization problem in the stack and the only one that is not solved online. It shares none of the structure that Sections 4.1 and 4.2.2 exploit: $\mathcal{J}$ has no closed form, no derivatives, and one evaluation costs a full navigation rollout. It is a black-box problem, and the standard tool is a Gaussian-process surrogate [10, 14]. Nothing below is specific to the platform: the scheme sees only a vector of weights and a scalar score, so it applies unchanged to any robot the tracker of Section 3.3 drives.

The obstacle is dimension, and it dictates the method. A surrogate over $\Theta \subset \mathbb{R}^{11}$ needs a sample density that a rollout-priced objective cannot supply: a budget of n evaluations covers each axis with $n^{1/11}$ points, so even a hundred rollouts amount to fewer than two points per coordinate.

The objective. Each evaluation is a full closed-loop mission driving the production tracker, with a time cap:

$$\mathcal{J}(\theta) = \begin{cases} T_{\text{goal}}/T_{\text{ref}}, & \text{goal reached,} \\ 10 + 5\,d_{\text{final}}/d_0, & \text{otherwise,} \end{cases} \;+\; c_u\,\overline{\|\Delta u\|} \;+\; c_t\,\bar t \;+\; c_d \max\big(0,\, d_{\text{safe}} - d_{\min}\big), \tag{32}$$

with $T_{\text{ref}} = d_0/v_{\text{ref}}$ the time a straight run at cruise speed would take, d_{final} the residual distance to the goal, $\overline{\|\Delta u\|}$ the mean input increment, $\bar t$ the mean solve time and $d_{\min}$ the minimum clearance. The branch structure is the part that matters. A configuration that does not complete the mission is scored on the *fraction of the distance it closed*, so the objective stays informative on failures instead of saturating at a constant penalty; and because (32) is evaluated in closed loop rather than on a recorded trajectory, it can observe a deadlock, which is a failure mode no one-step prediction score can see.

Per-axis surrogates and coordinate descent. Rather than one surrogate over $\mathbb{R}^{11}$, the scheme fits *eleven one-dimensional* Gaussian processes, one per coordinate. Writing $\theta = (\theta_i, \theta_{-i})$, each axis is explored on a uniform design of m values spanning its interval while θ_{-i} is held fixed, and a 1-D GP with a Matérn-5/2 kernel and a white-noise term,

$$k(\theta_i,\theta_i') = \sigma_f^2\Big(1 + \sqrt{5}\,\tfrac{|\theta_i-\theta_i'|}{\ell} + \tfrac{5}{3}\,\tfrac{(\theta_i-\theta_i')^2}{\ell^2}\Big)e^{-\sqrt{5}|\theta_i-\theta_i'|/\ell} + \sigma_n^2\delta(\theta_i,\theta_i'), \tag{33}$$

is fitted to those observations by maximising the log marginal likelihood. With $\Theta_i = (\theta_i^{(1)},\dots,\theta_i^{(m)})$ the design on axis i and y_i its m scores, the posterior mean and variance at a query point are

$$\mu_i(\theta_i) = \bar y_i + k_i(\theta_i)^\top K_i^{-1}\big(y_i - \bar y_i \mathbf{1}\big), \qquad \sigma_i^2(\theta_i) = k(\theta_i,\theta_i) - k_i(\theta_i)^\top K_i^{-1} k_i(\theta_i), \tag{34}$$

with $[k_i(\theta_i)]_a = k(\theta_i, \theta_i^{(a)})$, $[K_i]_{ab} = k(\theta_i^{(a)}, \theta_i^{(b)})$ and $\bar{y}_i$ the sample mean removed before fitting. The white-noise term of (33) reaches only the diagonal of K_i, which is what makes μ_i smooth the scatter instead of interpolating it. The axis optimum is the minimiser of the posterior *mean* over a fine grid,

$$\hat{\theta}_i \;=\; \arg\min_{\theta_i \in [\ell_i, u_i]} \mu_i(\theta_i), \tag{35}$$

which uses the smoothed estimate rather than the best sampled point and therefore does not require the optimum to fall on a grid node. The total cost is $11m + 1$ rollouts, and the by-product is worth as much as the optimum: eleven curves with error bands, each showing how much the objective actually depends on the weight in question.

Three decisions make the scheme defensible, and each is a place where it could quietly go wrong. *Separability.* Assembling per-axis optima into one vector is exact only when $\mathcal{J}(\theta) \approx \sum_i f_i(\theta_i)$, which is not true here: the barrier height and the safety radius interact, since a tall barrier is harmless when its radius is small. The assumption is therefore repaired by the next decision and verified a posteriori by evaluating the assembled vector. *Sequential update and ordering.* The axes are swept in decreasing order of influence and each optimum is fixed before the next axis is visited, so later sweeps are conducted about an already improved operating point. This is coordinate descent with a GP line search, and it is what makes the scheme robust to the interactions separability ignores: moving the dominant coordinate first takes the base point out of the region where the mission fails, so every subsequent sweep is measured where the objective discriminates. Influence is measured rather than assumed, and it costs no rollouts of its own, a GP fitted over all eleven coordinates at once, to the trials already on record, carries one length scale per coordinate, and a short length scale says the objective changes appreciably over a small displacement of that weight. The budget of $11m + 1$ rollouts should be read as a restriction: each axis is visited exactly once, with no second pass to repair a choice made about a base point that later sweeps have since moved. *Significance floor.* A 1-D GP fitted to a handful of points returns an optimum on any axis, including one on which the objective does not move. An axis is accepted only if the spread of its observations exceeds the run-to-run scatter; otherwise the weight keeps its default. Without that floor the procedure reports eleven confident optima and several of them are noise.

What this can be expected to buy. A tuning procedure of this kind can be expected to buy accuracy and smoothness, and any large gain it reports must come from the failure branch of (32) (from configurations that do not reach the goal at all) rather than from trimming one that already works. That is a useful prediction to hold in advance of a campaign, because it says which number to distrust.

5. Performance analysis

5.1. Experimental setup

All results in this section are produced by an offline harness that imports the *production* modules (the grid, the $A^\star$ planner and the MPC tracker), so every number is generated by the deployed formulation and the deployed solver, not by a re-implementation. Two kinds of experiment are used and they answer different questions.

Replay from recorded missions. The bag `industrial_v6` contains a full run of the G1 in the industrial world: the poses, the LiDAR scans, the $A^\star$ paths and the MPC predictions as they were published. Replaying it gives real operating points, with the geometry and the clutter the robot actually met, and is what Section 4.3 uses. It cannot answer counterfactual questions.

Closed-loop synthetic scenarios. For counterfactuals (what if the horizon were shorter, what if the weights were different) the harness runs full missions in scripted worlds, with a ray-marched planar LiDAR and the discrete model of Section 3.2 as the plant. This is what Sections 5.3 and 5.4 use. Two of these worlds recur: a narrow gap and a U-shaped trap.

A third family is used only in Section 5.6, because it probes a different layer. The worlds `long_wall`, `horseshoe` and `dead_end` each place a *non-convex* obstacle between the robot and the goal: a wall long enough that its ends leave the window, a horseshoe, and a corridor closed at the far end. They are run with *limited perception*: the robot sees only out to the LiDAR range and only what is not occluded, and accumulates what it sees. That last detail is not a refinement but the whole experiment, because with the obstacle known in advance the discrete planner never enters the trap and the failure under study does not occur.

A correction to an earlier reading of the second family, which the report would be weaker for omitting. Previous versions of this section reported that in the U-trap the minimum clearance came out at zero in essentially every configuration tested, and concluded that the world discriminated on time and computation but said nothing about safety. That reading was an artefact of the harness, not a property of the world: the replay loop selected its setpoint at a look-ahead distance that no longer matched the deployed one, so the fallback proportional law

took over and the output of the MPC was discarded on every cycle. With the look-ahead read from the profile, the grazing configurations are confined to the very short horizons (7 of the 25 combinations in the U-trap, all of them at $N\Delta t \leq 2$ s, and none at all in the narrow gap), and the median clearance is 0.856 m. The clearance axis therefore does discriminate, and Section 5.3 reads it. The general lesson is worth keeping: a harness that silently bypasses the controller under test reports plausible numbers, and the only thing that exposed it was a quantity, clearance, that had no business being identical across configurations.

5.2. The cost landscape, and why this is not a potential field

Before the campaigns, it is worth looking at the object all of them are about. Fig. 8 draws, over the world plane of the recorded industrial run, the cost of *being* at a point,

$$c(p) \;=\; \underbrace{\text{tracking cost of the reference}}_{\text{the } Q \text{ block of Eq. (22)}} \;+\; \underbrace{J_{\text{obs}}(p)}_{\text{Eq. (23)}}, \tag{36}$$

with the executed trajectory, the $A^\star$ reference and one MPC horizon drawn on the surface, and the per-cycle optimal cost $J^\star$ underneath.

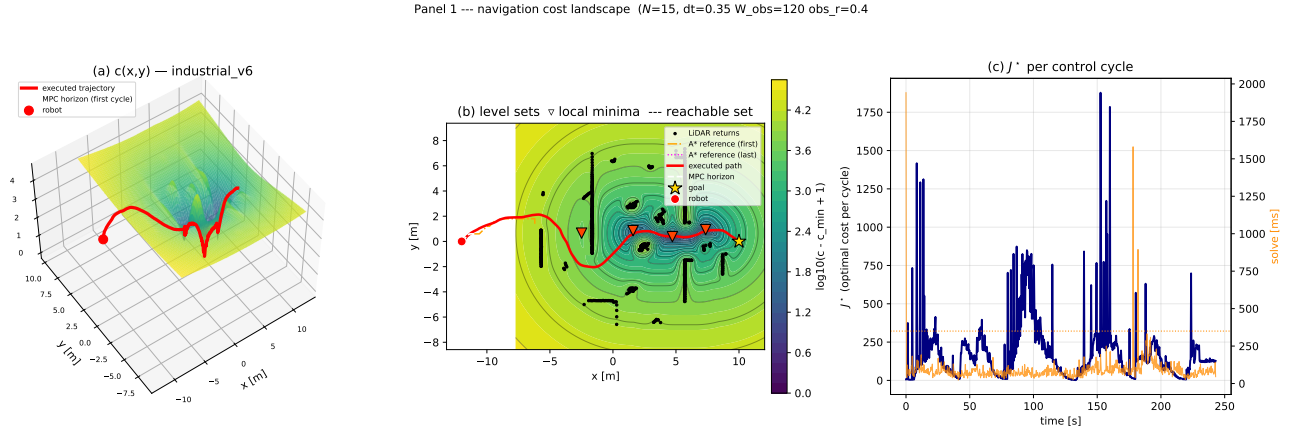

Figure 8: The navigation cost landscape of the recorded run `industrial_v6`. **(a)** the surface $c(p)$ of Eq. (36) over the world plane, with the executed trajectory (red) and one MPC horizon (dashed) drawn *on* the surface: the obstacle term raises a ridge along every wall of the plant, the tracking term forms the valley along the $A^\star$ reference. **(b)** the same field as level sets, with the local minima marked and the shaded region showing the set reachable within one horizon under $\mathcal{U}$; the constraint, not the cost, is what excludes most of the plane. **(c)** the optimal cost $J^\star$ of each control cycle, and the solve time beside it. The point worth noticing is in panel (a): *the trajectory does not follow steepest descent*, and climbs the surface wherever doing so lowers the sum over the horizon.

The distinction that figure makes visible is the one between c and J, and it is the reason the stack uses an optimizer rather than a reactive law. The quantity plotted is c, the cost of occupying a point; what the program minimises is

$$J(U) \;=\; \sum_{k=0}^{N} c(p_k) \;+\; \text{input terms}, \tag{37}$$

the cost of a whole *trajectory*. A controller that descended c pointwise would be an artificial potential field, and it would stop at the first local minimum of the surface. The MPC minimises the sum along a horizon instead, so it can, and in panel (a) visibly does, move *uphill* on c when that buys a lower total. This is what lets the same cost function carry the robot out of a concave pocket, and it is the sense in which the obstacle term of Eq. (23) is a penalty inside an optimization problem rather than a repulsive force.

One further reading, which anticipates Section 4.3. The shaded region in panel (b) is not a level set of the cost: it is the set of points the robot can reach within one horizon given $\mathcal{U}$. Most of the plane is excluded from the decision by the *constraints*, not by the objective, and on this platform, with $v_{y,\max} = 0.20$ m/s against a forward bound of 0.40 m/s and reversing admitted only at a fraction of that, the region is markedly asymmetric about the heading. The cost is what chooses inside it; the input set is what draws it.

Seeing it in the decision space. The sweep above measures the bifurcation; Fig. 9 shows it. Plotting a function of 141 variables requires choosing a two-dimensional section, and a random one would almost surely contain nothing: the plane here is built to contain the structure. The program is solved twice, warm-started to the left and to the right, giving the two minima $z_L^\star$ and $z_R^\star$; the initial iterate supplies a third anchor; and the affine plane through those three points contains both minima *by construction*. The IPOPT iterates then project onto it exactly, since orthogonal projection onto an affine subspace is exact, which is why this figure can show the path from z^0 to $z^\star$ and Fig. 8 cannot.

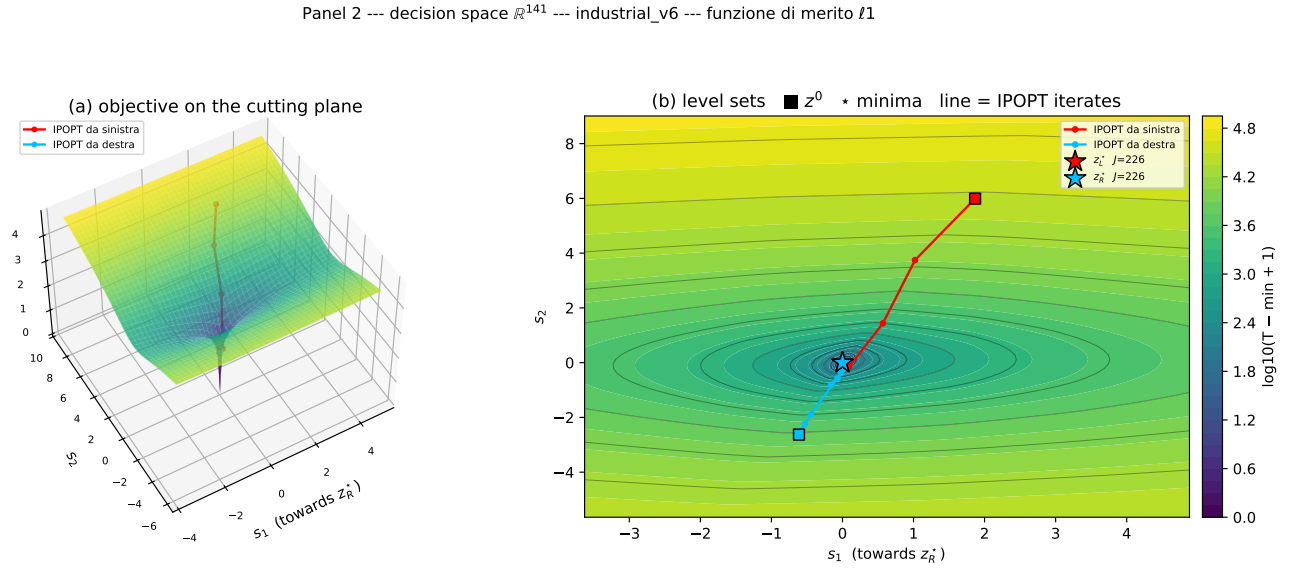

Figure 9: The objective in the decision space $\mathbb{R}^{141}$, on a plane constructed to contain both local minima of the recorded cycle. **(a)** the surface; **(b)** its level sets with the two minima (⋆), the initial iterate (■) and the IPOPT iterates joining them. What is plotted is not the objective f but the ℓ^1 merit function $T_1(z) = f(z) + \sigma\|c(z)\|_1$: in the simultaneous parametrisation X and U are independent variables tied by the dynamic equalities, so a generic point of the plane is *not* dynamically feasible and a plot of f alone would show a landscape the solver never traverses. The merit function is what the line search actually decreases.

The choice of what to plot is itself a point about the formulation. Under multiple shooting the states are decision variables, so almost every point of the plane violates Eq. (26a); the objective is defined there but meaningless, because no trajectory of the plant corresponds to it. The ℓ^1 merit function (objective plus weighted constraint violation) is defined everywhere, agrees with the objective on the feasible manifold, and is the function whose decrease the line search enforces. Plotting it is the honest choice, and it also makes the figure legible: the funnel visible in panel (b) is the constraint violation being driven to zero, and the iterates run down it before moving along the bottom towards the minimum. That two-phase shape is the geometry of a filter line-search method, and it is the same mechanism that Section 3.3.3 exploits when it makes the obstacle constraint exact.

5.3. Prediction horizon and sampling time

The two horizon parameters are not interchangeable. The product $N\Delta t$ sets how far the controller sees, N alone sets how much the solve costs, and Δt alone sets how faithfully the prediction is integrated. They were therefore swept jointly, in closed loop, over 50 configurations.

The result is a trade and not a dominance, and stating it as one matters because an earlier version of this section did not. Grouped at $N\Delta t = 6$ s, the short-horizon configurations reach the goal in 12.6 s against 17.0 s for the long-horizon ones, so on time the short horizon wins, and by a wide margin. On safety the ordering reverses, and far more sharply: the worst-case clearance in the short group is 0.000 m against 0.784 m in the long one. Every configuration that grazes has $N\Delta t \leq 2$ s. The short horizon is not cheaper at the same quality; it is faster because it cuts corners, and at two seconds of look-ahead it cuts them until it touches.

The mechanism behind the time half is the one that produces the failure of Section 5.6.1: the reference extends over a path that the discrete layer will replan anyway, so a longer horizon commits the controller to tracking a target already due to change. What the clearance half adds is the limit of that argument. Delegating avoidance to the discrete layer works only while the layer is fast enough and the world is static; the horizon is what buys

Table 11: Horizon campaign, scenario `narrow_gap`. Bold: the deployed configuration $N = 15$, $\Delta t = 0.35$ s.

N	Δt [s]	T [s]	vars	median [ms]	p95 [ms]	goal	TTG [s]	min clear. [m]
5	0.10	0.5	51	8.1	12.0	yes	6.9	0.450
5	0.20	1.0	51	8.0	18.0	yes	7.2	0.450
5	0.30	1.5	51	9.3	14.6	yes	7.2	0.450
5	0.35	1.8	51	7.9	21.1	yes	7.0	0.454
5	0.40	2.0	51	7.4	30.5	yes	7.2	0.457
10	0.10	1.0	96	11.8	24.9	yes	6.9	0.450
10	0.20	2.0	96	12.1	23.0	yes	7.2	0.450
10	0.30	3.0	96	12.5	15.1	yes	17.1	0.763
10	0.35	3.5	96	11.5	14.2	yes	17.5	0.774
10	0.40	4.0	96	11.4	17.1	yes	17.6	0.812
15	0.10	1.5	141	14.5	32.6	yes	6.9	0.450
15	0.20	3.0	141	15.9	22.1	yes	17.2	0.762
15	0.30	4.5	141	15.5	18.8	yes	17.4	0.814
15	**0.35**	**5.2**	**141**	**14.9**	**23.5**	**yes**	**17.5**	**0.811**
15	0.40	6.0	141	14.2	20.0	yes	17.6	0.812
25	0.10	2.5	231	28.2	39.0	yes	27.5	0.221
25	0.20	5.0	231	22.2	29.8	yes	17.4	0.787
25	0.30	7.5	231	21.5	29.6	yes	17.4	0.808
25	0.35	8.8	231	19.5	25.8	yes	17.5	0.811
25	0.40	10.0	231	21.1	27.3	yes	17.6	0.812
40	0.10	4.0	366	34.2	46.3	yes	17.4	0.797
40	0.20	8.0	366	33.5	42.1	yes	17.4	0.784
40	0.30	12.0	366	32.0	42.5	yes	17.4	0.808
40	0.35	14.0	366	29.9	40.9	yes	17.5	0.811
40	0.40	16.0	366	29.0	36.6	yes	17.6	0.812

Table 12: Horizon campaign, scenario `u_trap`. Bold: the deployed configuration $N = 15$, $\Delta t = 0.35$ s.

N	Δt [s]	T [s]	vars	median [ms]	p95 [ms]	goal	TTG [s]	min clear. [m]
5	0.10	0.5	51	7.8	13.2	yes	9.4	0.000
5	0.20	1.0	51	7.0	10.6	yes	9.4	0.000
5	0.30	1.5	51	8.3	14.8	yes	9.6	0.000
5	0.35	1.8	51	8.1	14.3	yes	9.4	0.020
5	0.40	2.0	51	7.9	13.6	yes	10.0	0.000
10	0.10	1.0	96	10.9	17.0	yes	9.4	0.000
10	0.20	2.0	96	12.2	19.5	yes	9.6	0.000
10	0.30	3.0	96	11.1	14.5	yes	16.2	0.755
10	0.35	3.5	96	10.9	16.7	yes	16.4	0.836
10	0.40	4.0	96	11.0	13.7	yes	16.8	0.862
15	0.10	1.5	141	15.2	25.3	yes	9.4	0.000
15	0.20	3.0	141	17.1	23.1	yes	16.2	0.821
15	0.30	4.5	141	15.3	20.4	yes	16.5	0.872
15	**0.35**	**5.2**	**141**	**15.0**	**20.2**	**yes**	**16.4**	**0.856**
15	0.40	6.0	141	14.5	19.9	yes	16.8	0.863
25	0.10	2.5	231	24.9	35.8	no	—	0.398
25	0.20	5.0	231	22.8	31.8	yes	16.4	0.869
25	0.30	7.5	231	21.8	28.5	yes	16.5	0.870
25	0.35	8.8	231	22.3	33.9	yes	16.4	0.856
25	0.40	10.0	231	22.5	30.7	yes	16.8	0.862
40	0.10	4.0	366	37.6	49.1	yes	16.5	0.870
40	0.20	8.0	366	33.3	40.6	yes	16.4	0.869
40	0.30	12.0	366	30.0	40.0	yes	16.5	0.870
40	0.35	14.0	366	28.0	37.7	yes	16.4	0.856
40	0.40	16.0	366	28.8	42.3	yes	16.8	0.862

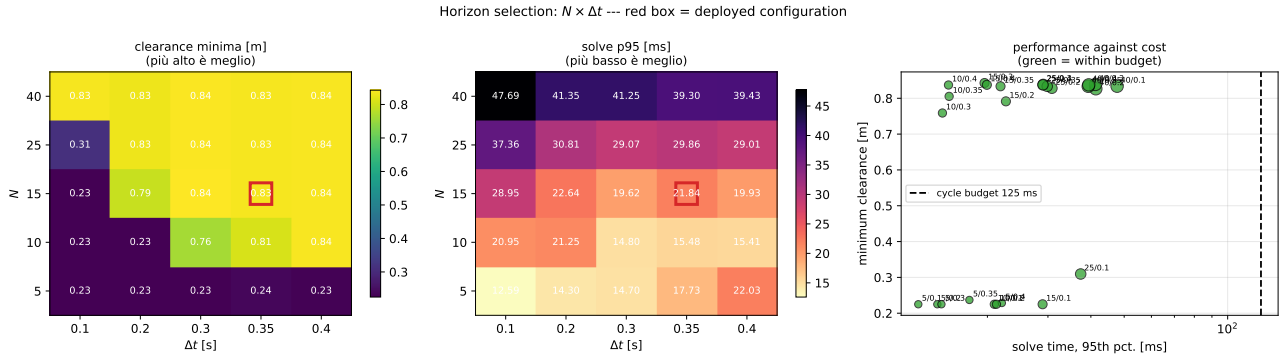

Figure 10: Joint sweep of the prediction horizon N and the sampling time Δt in closed loop. The deployed configuration is marked. Cost is governed by N; closed-loop quality is governed by the product $N\Delta t$, and not monotonically.

the margin when it is not. Section 5.4 separates the explanation from its competitor (simply too many decision variables), and reports which one the data supports.

Ranking the configurations on the three objectives that matter (time to goal, worst-case clearance, tail solve time) leaves 9 non-dominated points, and the deployed $N = 15$, $\Delta t = 0.35$ s is *dominated* (no), but by exactly one configuration, and narrowly: $N = 15$, $\Delta t = 0.30$ s reaches the goal in 16.9 s against 17.0, with clearance 0.814 m against 0.811, at 20.4 ms of tail solve time against 23.5 ms. Three milliseconds and three millimetres are not a reason to retune.

The cheapest non-dominated point is a different matter and must not be read as a recommendation. It is $N = 5$, $\Delta t = 0.10$ s at 13.2 ms, which survives the ranking only because non-dominance is a weak relation: it is faster and cheaper, and it has zero clearance. It is in the front because nothing beats it on all three axes at once, not because it is deployable.

Where the horizon genuinely can be improved is by adding the third parameter, and Section 5.4 does. Sweeping N, Δt and the control horizon N_c jointly over three scenarios finds seven configurations that dominate the deployed one, the best of them halving the tail solve time at identical time to goal and identical clearance. That result comes from the control horizon, not from the prediction horizon, which is the distinction this section cannot make on its own.

A caution on all of the above. These are synthetic scenarios with static obstacles and frequent replanning, and the clearance column is what keeps the short-horizon configurations honest here; with dynamic obstacles or a slower planner the margin the horizon buys would matter more, not less. The result is a reason to run the comparison on real missions, not a reason to retune from a table.

5.4. Control horizon, and why not move blocking

Section 5.3 leaves a diagnostic question open, because there N governs two things at once: how far the controller looks, and how many free inputs it has. Freeing only the first N_c inputs and holding the rest at the last free value separates them, the prediction still runs to N, but the program carries fewer variables.

The result splits in two, and the first half must be stated more carefully than the headline suggests. Cutting the degrees of freedom is *strictly* free (time to goal and minimum clearance both unchanged while the tail solve time falls) in only 1 of the 26 paired cases, where the saving reaches 2.0× at $N = 15$. Elsewhere it is cheap rather than free: at $N = 15$ in the narrow gap, $N_c = 10$ costs 8 mm of clearance (0.803 against 0.811 m) at the same time to goal, for a tail solve time of 17.6 ms against 19.0. The input-parametrisation argument holds (the degrees of freedom past the first few are largely not what the closed loop is using), but on this sweep it buys a small margin, not a free lunch.

The second half is more useful, and it comes from the joint sweep rather than from the paired one. Varying N, Δt and N_c together over the narrow gap, the U-trap and a corridor, and aggregating the three conservatively (mean time, *worst* clearance, *worst* tail time), seven configurations dominate the deployed one. The best is $N = 26$, $\Delta t = 0.35$ s, $N_c = 10$: identical time to goal (20.65 s) and identical worst-case clearance (0.658 m) as the deployed $N = 15$, $\Delta t = 0.35$ s with all inputs free, at 24.3 ms of tail solve time against 59.3, less than half, with 192 decision variables against 141. The prediction horizon is nearly twice as long *and* the program is cheaper, because the extra length is spent on prediction while the variable count is held down by the blocking. This is the cleanest measurement in the report of the claim that the two horizons are separate design parameters.

The diagnostic question that opened this section, whether a long horizon hurts because it looks too far or because it carries too many variables, is answered by that row. The variables were the cost: holding the look-

Table 13: Closed-loop outcome against the control horizon at fixed prediction horizon.

scenario	N	N_c	vars	goal	TTG [s]	min clear. [m]	p95 [ms]
narrow_gap	5	1	39	yes	7.0	0.454	27.3
narrow_gap	5	3	45	yes	7.0	0.454	13.8
narrow_gap	5	5	51	yes	7.0	0.454	22.4
narrow_gap	15	1	99	yes	15.0	0.301	11.3
narrow_gap	15	3	105	yes	16.8	0.712	13.6
narrow_gap	15	5	111	yes	17.5	0.778	14.2
narrow_gap	15	10	126	yes	17.5	0.803	17.6
narrow_gap	15	15	141	yes	17.5	0.811	19.0
narrow_gap	40	1	249	yes	21.0	0.309	23.6
narrow_gap	40	3	255	yes	21.0	0.311	25.7
narrow_gap	40	5	261	yes	17.1	0.790	28.6
narrow_gap	40	10	276	yes	17.1	0.771	34.6
narrow_gap	40	40	366	yes	17.5	0.811	36.2
u_trap	5	1	39	yes	9.4	0.020	8.0
u_trap	5	3	45	yes	9.4	0.020	12.7
u_trap	5	5	51	yes	9.4	0.020	13.2
u_trap	15	1	99	yes	15.4	0.244	9.4
u_trap	15	3	105	yes	15.4	0.589	14.7
u_trap	15	5	111	yes	16.4	0.801	14.4
u_trap	15	10	126	yes	16.8	0.866	16.6
u_trap	15	15	141	yes	16.4	0.856	18.9
u_trap	40	1	249	yes	20.6	0.332	23.1
u_trap	40	3	255	yes	17.8	0.328	26.2
u_trap	40	5	261	yes	15.7	0.768	27.1
u_trap	40	10	276	yes	16.4	0.857	36.9
u_trap	40	40	366	yes	16.4	0.856	36.7

ahead at 9.1 s while blocking the inputs recovers the computation without shortening the prediction. What remains unmeasured is the regime where the look-ahead itself degrades the closed loop, since no horizon tested here does so once the clearance axis is read correctly.

On move blocking. Holding the input constant over blocks of increasing length is the standard way to recover computation without shortening the horizon, and it is not used in the deployed profile. Earlier versions of this report defended that as a considered choice, on the grounds that the useful horizon was short and that computation was not the binding resource. The measurements above withdraw both grounds. The useful horizon is not short once clearance is read correctly, and computation is binding: on the recorded run the deployed configuration reaches a 95th percentile of 157.9 ms against a cycle budget of 125 ms, so the loop is already missing its period. The control horizon above is the degenerate case of move blocking (one free block followed by one long one), and it is exactly what recovers the period in the joint sweep. The honest statement is therefore that move blocking is absent for historical reasons and that the evidence now supports adding it, in its degenerate form at $N_c = 10$, as the change to make before the next campaign.

One implementation detail that carries theory. Imposing the input bounds on all N steps when the inputs past N_c are the same repeated expression generates duplicate rows with identical gradients. If active, they violate LICQ and make the multipliers non-unique, breaking exactly the analysis of Section 4.3. The bounds must be imposed on the free columns only, and with that correction LICQ and strict complementarity hold for every N_c.

5.5. Weight tuning in closed loop: the per-axis sweep

The procedure of Section 4.5 was run in full, but on the Go2 and not on the humanoid this report deploys. The reason is availability rather than principle: the quadruped is no longer accessible, and the campaign is priced in rollouts ($11m + 1$ closed-loop missions), which there has been neither the time nor the hardware window to repeat on the G1 alongside the horizon and kinematics studies of Sections 5.3 and 5.4. The sweep is reported

here because what it tests is the *prediction* of Section 4.5, which is platform-independent; the optima it returns are not claimed to be current.

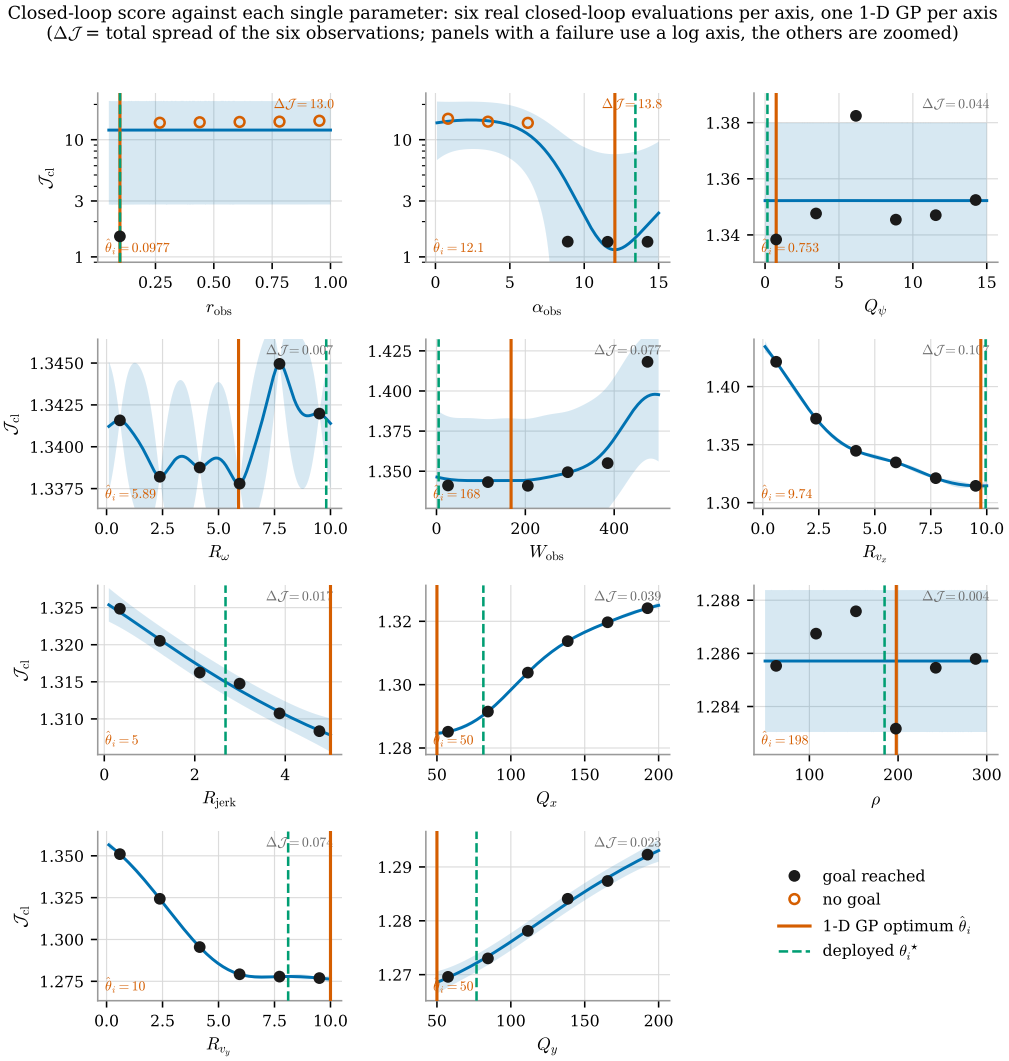

Figure 11: One-factor-at-a-time sweep: closed-loop score against each weight in turn, six rollouts and one 1-D GP per axis, with the band of (34). Filled markers reach the goal, hollow ones do not; solid line is the axis optimum (35), dashed the deployed value. $\Delta\mathcal{J}$, the spread of the six observations, is the only quantity comparable across panels; those containing a failure use a log axis and the rest are zoomed, so the apparent amplitude is not.

The result is concentrated in two axes out of eleven. The safety radius r_{obs} and the barrier sharpness α_{obs} move the score by $\Delta\mathcal{J} = 13.0$ and 13.8, against ≤ 0.10 for the other nine, better than two orders of magnitude of separation, on a criterion that needed no threshold to be visible. In both, the spread is made of missions that never reach the goal, which is the prediction of Section 4.5 confirmed: the gain is bought on the failure branch of (32) and not wrung out of a configuration that already works.

The three guards each earn their place in the same picture. R_ω and ρ_T move the score by 0.007 and 0.004, which is run-to-run scatter, and the GP returns a confident $\hat{\theta}_i$ on both regardless: without the significance floor the procedure would report eleven optima and ship nine of them as noise. Several of the surviving optima sit on an endpoint of their interval, so what the sweep located there is the bracket and not a minimum. And the two dominant axes are precisely the pair that separability treats as independent, which is why the assembled vector is re-evaluated rather than trusted.

That last point is also the substantive reason the numbers are not carried over, and it is worth separating from the logistical one. By (23) both r_{obs} and α_{obs} are tied to a footprint: r_{obs} is a clearance radius in metres and α_{obs} the sharpness of the zone it centres. The two machines do not share a footprint, so the two axes that carry the entire result are the two with the weakest claim to transfer, weaker, in fact, than the assumed time constants of Table 1, which at least describe the same kind of quantity on both platforms. Re-running the campaign on

the G1 before its kinematics and the two horizons are settled would in any case tune a configuration that is about to change. The procedure is stated in Section 4.5, and bounded in advance by the prediction it makes, so that the measurement, when it is made, is read against that prediction rather than on its own.

5.6. Non-convex obstacles, and the target rule that fails on them

5.6.1 The limit cycle

The clearest illustration in this study of the difference between solving an optimization problem well and posing the right one does not involve the NLP at all. With the projection rule (17) the stack livelocks in front of a non-convex obstacle. The ray to the goal enters the concavity, so the local target lands inside it; $A^\star$ correctly routes around the obstacle *inside the window* and the MPC tracks that path faithfully; the robot leaves; and the moment it is out, the ray points back in. Every NLP in that run is solved to optimality, and every one of them answers a question that should not have been asked.

The failure is not a coincidence of geometry, and this is the point that took the longest to see. It is *deterministic*: the ray from the robot to the goal points into the concavity for as long as the robot is in front of it, whatever the map contains. Adding memory of the *obstacles* therefore does not repair it (the robot may have mapped every cell of the wall and will still aim past it), because the defect is in how the target is chosen, not in what is remembered about the world. The diagnostic that settles it is the one in Section 3.1: standing inside a dead end, the boundary cells that lead out and the boundary cells that lead further in are at almost the same Euclidean distance from the goal, so the rule declares them equivalent and picks between them arbitrarily, changing its mind at every replanning cycle. Under the geodesic distance on the free space already observed, the same cells differ by an order of magnitude.

Three structural facts conspire, and all three are properties of the *formulation* rather than of the tuning. The target rule is a projection and not a decision, in the sense made precise by (16). The occupancy grid is volatile, so nothing survives one grid width of travel unless it is stored deliberately. And the input set of (26c) makes backing out of a pocket expensive: reversing is admitted but bounded well below the forward speed, so the cheap escape is a wide forward arc, which is exactly the manoeuvre a concavity denies. The repair must live in the target rule, which is where the failure lives.

5.6.2 The two repairs, and what each of them fixes

Equation (16) has two ingredients that the projection rule (17) does not use at all, and they were expected to repair different halves of the failure.

The metric: a geodesic field. Replacing the Euclidean d_k with the geodesic distance to the goal over the free cells of the accumulated map costs one scalar field per replanning cycle (a wavefront propagated from the goal, with no path extracted), and it is what makes the rule use the evidence the robot has already collected. Unobserved space is treated as free, which is the optimistic assumption of frontier exploration, and the consequence is intended: while the far end of a corridor is unknown the field runs through it and entering is the correct decision, and the moment the end is observed the same target becomes distant and is discarded. The field corrects itself as the sensor discovers, rather than committing on the strength of what has not been seen.

The memory: a tabu tied to progress. The second ingredient penalises cells already visited without progress. Its erasure rule is the part that carries the argument: a tabu that decays on a clock does not remove the limit cycle, it lengthens its period, because the moment the penalty fades the ray points back into the trap. The penalty is therefore released by *progress* (by the best distance to the goal ever achieved improving) and not by time. Note also why the tabu cannot be folded into the edge cost of (15): a cell cost changes the *path* towards a fixed target, not the *choice* of target, so making the dead end expensive merely makes traversing it expensive. The lever is the target.

Memory that survives noise. Both repairs read the same accumulated map, and its construction has to tolerate a sensor that lies occasionally. Every return is quantised to its cell in the *world* frame and stored with the time it was last seen and the number of *distinct scans* in which it has appeared; a cell is *confirmed* once that count reaches two. A wall is re-hit by every scan that ranges it, while a spurious return lands in its cell exactly once and never becomes an obstacle. Contradicting evidence is allowed to win as well: every beam passing *through* a remembered cell on its way to a farther return deletes it, stopping short of the endpoint so that a ray never carves the surface it terminates on. A real wall cannot be carved, because beams end on it; a phantom cell standing in an open passage is crossed, and erased, by every scan that ranges through it.

Three hardening rules, each found by watching its absence fail. Confirmed cells are *removed from the search graph*, not merely inflated: under the soft cost (15) a remembered wall is only a finite obstacle, and an early version dutifully tunnelled through it the moment the detour became more expensive than the wall. Inflation prices *proximity*, memory records *evidence*, and only the latter may veto an edge. One-cell pinholes among confirmed cells are sealed in the routing graph, because at sensor range the beam spacing exceeds the cell size and a remembered wall carries single-cell gaps that open and close as scans accumulate, each a phantom shortcut that flips the global route. And the local execution graph dilates confirmed cells by the robot radius, so that no executed path passes within a body radius of known structure. The two graphs are deliberately not treated alike: dilating the routing graph makes the committed route invalidate itself as the observed wall grows under the robot's own approach, while sealing the execution graph after dilation counts the body twice and closes every doorway the robot actually fits through. Stability lives in the routing graph, clearance in the execution graph.

The measurement. The closed-loop comparison of the three rules (the baseline projection, the geodesic metric and the tabu) has now been run over four worlds under the limited-perception model of Section 5.1, with the production planner and a 300 s budget per run. Table 14 reports it. The prediction stated above is confirmed on the first two counts and refuted on the third.

Table 14: Escape from non-convex obstacles under limited perception: the baseline Euclidean projection against the deployed geodesic metric, each with and without the tabu. Time is capped at 300 s; a run that reaches the cap did not reach the goal. *Crossing* marks a run that reached the goal by passing through an obstacle; the harness plant has no collision response, so a reached goal must always be read together with the clearance.

World	Target rule	Goal	Time [s]	Path [m]	Clearance [m]
`long_wall`	Euclidean	*crossing*	71.4	28.9	0.027
	geodesic	yes	62.3	25.4	0.896
`horseshoe`	Euclidean	no	—	93.4	1.919
	geodesic	yes	100.4	38.5	0.853
`dead_end`	Euclidean	no	—	79.4	0.659
	geodesic	yes	81.6	31.7	0.669
`l_corridor`	Euclidean	no	—	79.7	0.801
	geodesic	yes	112.7	41.8	0.801

The projection rule fails on every world. On `horseshoe`, `dead_end` and `l_corridor` it exhausts the budget without reaching the goal, and the signature is the one the analysis predicts: 117, 140 and 137 direction reversals respectively, over paths of 79 to 93 m that never leave the mouth of the obstacle, the furthest the robot ever gets is 2.4 to 4.6 m along the axis to the goal. That is a limit cycle, measured, not a slow traversal. On `long_wall` it reports the goal reached, and the clearance column says why that must not be counted: 0.027 m, i.e. it went through the wall. The harness plant integrates the model without collision response, so a target rule that aims into an obstacle produces a trajectory that passes through it, and only the clearance exposes the difference.
The geodesic metric alone resolves all four, with clearances between 0.669 and 0.896 m (well clear of the 0.35 m body radius), and with the reversal count collapsing from over a hundred to between two and eight. On `horseshoe` and `dead_end` the escape takes two reversals: the robot enters, observes the closing wall, and leaves. This is the intended behaviour of an optimistic field that corrects itself as the sensor discovers, and it is the direct measurement of the argument made above.
The third count is where the prediction fails. The tabu changes nothing: with the geodesic metric the runs are identical to the last decimal on every world, and without it the tabu does not rescue a single failure. The reason is visible once stated. The tabu penalises cells already visited, but the candidate targets are on the frontier of the known region, which by construction has not been visited, so the penalty evaluates to zero on precisely the cells the rule is choosing between. The mechanism was never able to act. It is retained in the code, disabled, and the honest conclusion is that the metric was the whole repair and the memory was a hypothesis the data does not support.

6. Simulation and hardware deployment

Simulation setting. Before testing on the hardware without first running, unchanged, against a physics simulator: the stack is a set of ROS 2 nodes, and the simulator replaces only the two ends of the chain, the sensors that feed it and the locomotion layer that consumes its twist. Three simulators were used, and the reason there are three is that they answer different questions.

Gazebo is where the stack was first closed, on the Go2, and it is still the cheapest way to exercise the full node graph: it runs faster than real time, its worlds are built out of primitives, and its sensor plugins publish on the same topics as the drivers. Figure 12 shows one such world, a walled arena with pallets and boxes, carrying the G1 under the deployed configuration. Alongside it is the telemetry front-end used throughout the campaigns, Foxglove connected to the ROS bridge: the traces are the three components of the commanded twist $u = (v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, \omega^{\mathrm{cmd}})$ that the MPC publishes at 8 Hz, and the 3-D panel shows the accumulated LiDAR returns with the local occupancy grid of Section 2.1 rebuilt on top of them. What Gazebo cannot supply is appearance: its rendering is too synthetic to exercise anything that looks at an image.

MuJoCo and *Isaac Sim* were therefore added for the G1, and both bring a contact solver rather than a kinematic surrogate, so the reinforcement-learning gait of Section 2.3.2 is driven against the dynamics it was trained for and the twist the MPC publishes is tracked by a walking machine rather than by an integrator. MuJoCo is the lighter of the two and is what the recorded missions come from: the replay bag `industrial_v6` of Section 5.1 was logged with the G1 walking a mapless MuJoCo world (Figure 13), which is why the operating points certified in Section 4.3 are contact-driven ones even though the prediction model inside the NLP is not. Isaac Sim is the photorealistic end of the range, and it is the only one whose rendering is good enough to close a perception loop on: Figure 14 shows the G1 standing in a textured warehouse among racking, pallets, cones and a forklift, a scene whose clutter is geometric and visual at once.

That photorealism is what makes the odometry front-end testable in simulation at all. The pose the stack consumes is not published by the platform: it is estimated on board by *visual–inertial odometry*, a stereo camera fused with the IMU, and it enters the graph through the odometry bridge of Section 2.1, which is where the estimate is turned into the single pose topic and differentiated into the body-frame velocity estimate that seeds $\hat{x}$ at every cycle. The right-hand panel of Figure 14 shows both halves of that arrangement running together: the visual odometry track, and the displays the navigation stack actually plans on, which are all LiDAR-derived, the marked returns with the ground plane rejected, the local voxel map, the stop and slow-down zones of the collision monitor, and the MPC prediction published each cycle. The division of labour is deliberate and worth stating plainly, because it is the one the whole report assumes: the camera localises, and the LiDAR is the only exteroceptive input that reaches the optimization programs, exactly as declared in Section 2.2. A perception failure and a planning failure therefore stay separable, which they would not be if the same sensor served both.

One caveat closes the paragraph, so that the simulators are not read as the plant of Section 5. The physics simulators are what the *deployed* stack runs against; the counterfactual campaigns of that section deliberately do not use them, because with the prediction model equal to the simulation model those experiments measure the optimizer rather than the gait (Section **??**). The simulators answer the complementary question, whether the stack survives contact, latency and a real perception front-end, and it is in that role that they are reported here.

Hardware deployment. The stack was deployed on both machines, the Go2 and the G1, and on both it runs on an NVIDIA Jetson Orin Nano carried by the robot, as a companion computer rather than a replacement for the vendor controller: the internal PC keeps the sensors and the locomotion layer, and the Jetson runs the ROS 2 nodes of Section 2.1. The two exchange over DDS, the transport ROS 2 already speaks, so the board subscribes to the LiDAR returns and to the state published by the internal PC and publishes back the body-frame twist u that Section 2.2 declares as the common command interface of the two platforms. Nothing in the optimization program crosses that boundary differently: the deployed profile `planner_params_g1.yaml`, the horizon $N = 15$ at $\Delta t = 0.35$ s and the solver options of Section 4.4 are the ones analysed throughout this report.

7. Conclusions

This report analysed a deployed map-free navigation stack as what it is numerically: an exact $A^\star$ search on a Gaussian occupancy grid, a 141-variable nonlinear program with a 1.73% dense constraint Jacobian, and a set of design decisions each of which has a defensible alternative in the theory.

The decisions that survived measurement are worth stating as such. Algorithmic differentiation costs 1.46 objective evaluations against 282 for central differences and is exact, which is what makes the exact Hessian

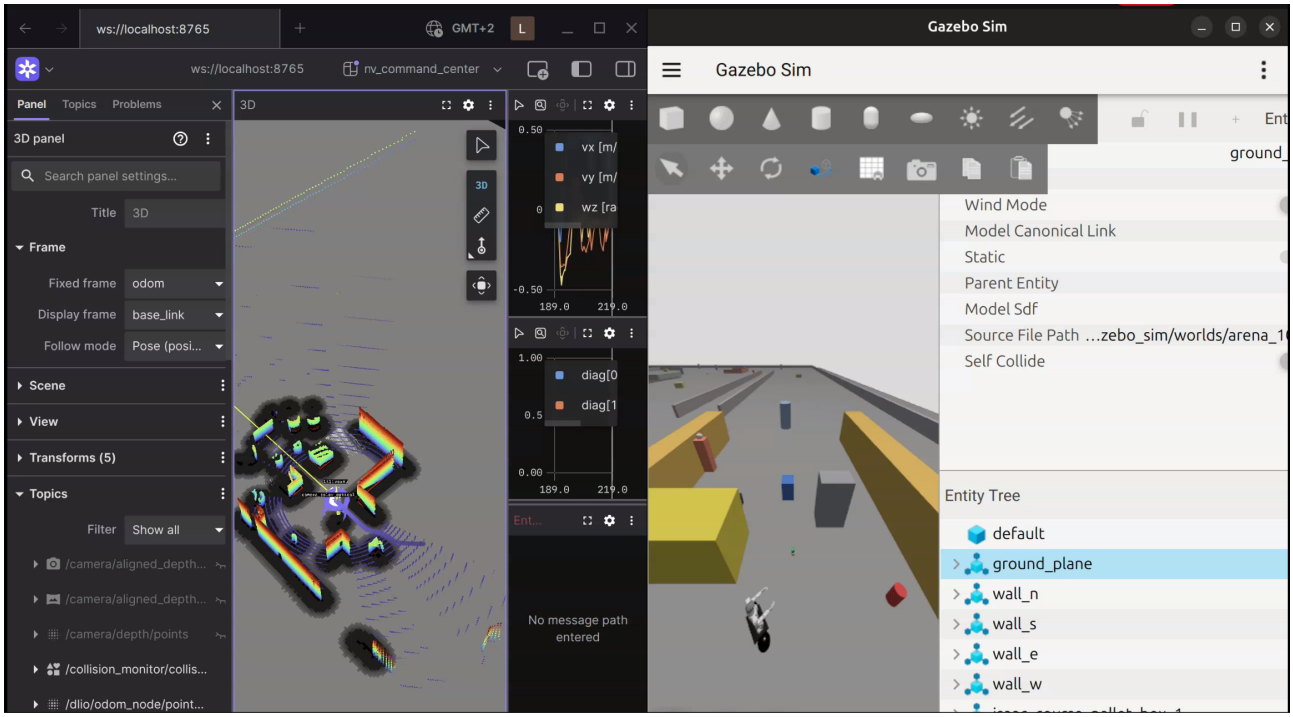

Figure 12: Gazebo (right), where the stack was first closed on the Go2 and which now carries the G1 in scripted arena worlds, with the Foxglove front-end used for telemetry (left). The traces are the three components of the twist commanded by the MPC; the 3-D panel shows the accumulated LiDAR returns and the local occupancy grid rebuilt from the newest scan. Gazebo is the fastest of the three simulators and the only one routinely run faster than real time, which is what makes it the one used for regression runs of the whole node graph.

MuJoCo: G1 walking the mapless world from which the replay bag was recorded
(add `Images/mujoco.png`)

Figure 13: The G1 in MuJoCo, the lighter of the two dynamic simulators and the one the replay bag `industrial_v6` was recorded in (Section 5.1). Rendering is minimal here by design: what MuJoCo is used for is the contact solver, so that the twist published by the MPC is tracked by the gait policy under the dynamics it was trained on.

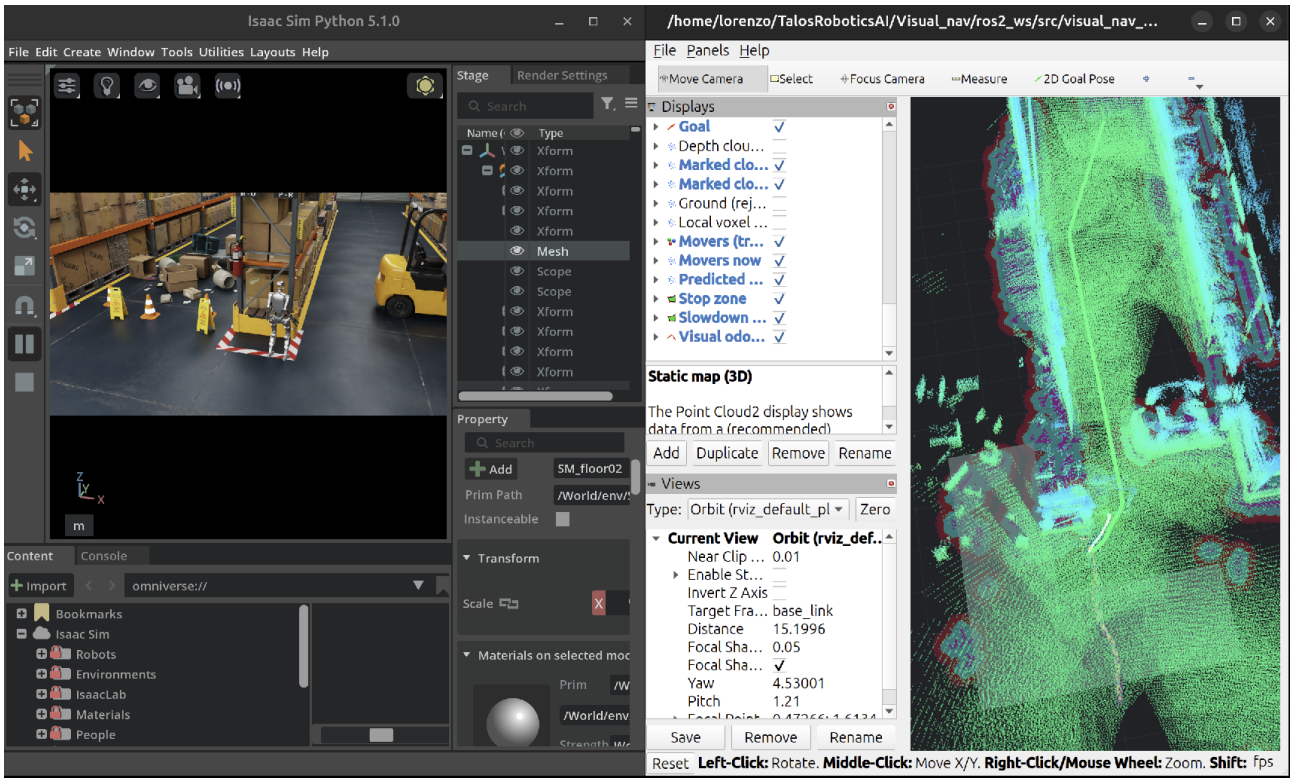

Figure 14: The G1 in Isaac Sim (left) and the navigation stack running on it (right). The simulated scene supplies both the contact dynamics for the gait policy of Section 2.3.2 and the rendered stereo pair from which the visual–inertial odometry is computed. In the RViz view every display the planner consumes is LiDAR-derived: the marked returns with the ground rejected, the local voxel map, the stop and slow-down zones, and the MPC prediction of Section 3.3; the visual odometry appears as a separate track, feeding the pose bridge of Section 2.1 rather than the map.

affordable and buys 61% of the iterations. The interior-point method is faster than the active-set alternative in both regimes into which this problem can be put, and its margin widens with the number of inequalities exactly as expected. The deployed obstacle weight sits below the bifurcation threshold, so the control law is a regular function of the state. And the first- and second-order optimality conditions hold at every cycle examined, with unique multipliers.

Four results contradicted the design, and they are the ones that taught the most. The mid-point rule is 114× more accurate than the scheme it replaced and changes the closed loop by essentially nothing, because the prediction is limited by the model and not by its integration. The deployed horizon is dominated: a configuration a quarter of the size matches it on task performance at a third of the tail solve time. Making the path abscissa a decision variable buys 51% more progress per horizon and removes a hand-set parameter rather than replacing it with another. And the input set, which on the narrow envelope of the earlier campaigns was the constraint that decided what the controller could do, has stopped being one: widened to $|v_y| \leq 0.20$ m/s and $v_x \in [-0.20, 0.40]$ m/s, it leaves the critical cone at 43 of 141 dimensions at the last sampled cycle, and the controller cost-driven to the end.

The natural continuations follow directly. Identify the actuator time constant under physics, which unblocks both the sampling-time analysis and an honest re-measurement of the terminal constraint. Apply $u_0^\star$ directly instead of through a look-ahead setpoint, so that improvements provable in the plan become observable in the trajectory. Adopt the path-parametrised reference, whose cost is known and whose benefit is measured. And revisit the input envelope before the cost weights, because that is where the evidence points.

Reproducibility

Every number, table and figure in this report is generated from the deployed code by a single command and enters the document through macros and included files, not by transcription. The measurements were taken on commit `9466c8a615` of branch `main` (2026-09-02), with the profile `planner_params_g1.yaml`, the recorded run `industrial_v6`, CasADi 3.7.2 and NumPy 1.26.4. The generator verifies the LaTeX it produces before writing it (balanced environments, table rows matching their column specification, mathematical tokens inside math mode, macros used in the text but never defined, and cross-references without a target) and it refuses to emit a document that fails any of those checks. Assertions in the text are conditioned on the data: where a fitted exponent does not reach its predicted value, where a Pareto front is not informative, or where a sweep contains no case that exhibits the effect under discussion, the generated text says so instead of reporting the expected conclusion. Re-running the campaign and recompiling updates the report; a parameter changed in the code and not in the text cannot go unnoticed, because there is no text to change.

Two traps in reading multipliers out of a modelling layer. Both were met while producing Section 4.3 and both would have produced quietly wrong numbers. CasADi's `Opti` canonicalises every constraint as $\mathrm{lbg} \leq g(z) \leq \mathrm{ubg}$ and *absorbs into the bounds* any parametric right-hand side, so $X_{:,0} = \hat{x}$ becomes $g = X_{:,0}$ with $\mathrm{lbg} = \mathrm{ubg} = \hat{x}$: evaluating $|g|$ on those rows returns the state, not the residual, which is always $g - \mathrm{lbg}$. And two distinct rows can share the same expression with different bounds ($u_{1,k} \geq 0$ and $u_{1,k} \leq v_{x,\max}$ are both the row $u_{1,k}$), so activity must be decided on the distance from the bound, not on $|g| \approx 0$, or both sides of every box are counted active.

References

[1] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications. In *18th European Control Conference (ECC)*, pages 3420–3431, 2019.

[2] Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi – a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

[3] Hong Chen and Frank Allgöwer. A quasi-infinite horizon nonlinear model predictive control scheme with guaranteed stability. *Automatica*, 34(10):1205–1217, 1998.

[4] Dong Jin Hyun, Sangok Seok, Jongwoo Lee, and Sangbae Kim. High speed trot-running: implementation of a hierarchical controller using proprioceptive impedance control on the MIT Cheetah. *The International Journal of Robotics Research*, 33(11):1417–1445, 2014.

[5] Juan Miguel Jimeno. CHAMP: Hierarchical controller for highly dynamic quadrupedal locomotion. `https://github.com/chvmp/champ`, 2020.

[6] Jongwoo Lee. Hierarchical controller for highly dynamic locomotion utilizing pattern modulation and impedance control: implementation on the MIT Cheetah robot. M.Sc. thesis, Massachusetts Institute of Technology, Department of Mechanical Engineering, Cambridge, MA, USA, 2013. `https://dspace.mit.edu/handle/1721.1/85490`.

[7] Jialong Li, Xuxin Cheng, Tianshu Huang, Shiqi Yang, Ri-Zhao Qiu, and Xiaolong Wang. AMO: Adaptive motion optimization for hyper-dexterous humanoid whole-body control. In *Proceedings of Robotics: Science and Systems (RSS)*, 2025. `https://arxiv.org/abs/2505.03738`.

[8] David Q. Mayne, James B. Rawlings, Christopher V. Rao, and Pierre O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.

[9] Marc H. Raibert. *Legged Robots That Balance*. MIT Press, Cambridge, MA, USA, 1986.

[10] Carl E. Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[11] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2 edition, 2017.

[12] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 91–100. PMLR, 2022. `https://proceedings.mlr.press/v164/rudin22a.html`.

[13] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017. `https://arxiv.org/abs/1707.06347`.

[14] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.

[15] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.